



UNIVERSIDAD
DE MÁLAGA

ESCUELA DE INGENIERÍAS INDUSTRIALES

Grado en Ingeniería en Tecnologías Industriales

TRABAJO DE FIN DE GRADO

Desarrollo de un Agente Autónomo para Logística de Almacenes mediante Aprendizaje por Refuerzo Profundo en Unity

AUTOR

Daniel Bustos Cano

TUTOR Y COTUTOR

Dr. Manuel Castellano Quero
Dr. Juan Antonio Fernández Madrigal

Departamento de Ingeniería de Sistemas y Automática

MÁLAGA · JUNIO 2026

[Esta página se mantiene en blanco voluntariamente]

Índice

Índice.....	3
1. Introducción	6
1.1. Contexto y Motivación	6
1.2. Objetivos del Trabajo.....	7
1.3. Alcance y Limitaciones.....	8
2. Marco Teórico y Estado del Arte.....	10
2.1. Aprendizaje por Refuerzo	10
2.1.1. Proceso de Decisión de Markov (MDP)	10
2.1.2. Métodos Policy Gradient	11
2.1.3. PPO (Proximal Policy Optimization).....	11
2.1.4. Consideración de DQN frente a PPO para el Control Discreto	12
2.2. Aprendizaje por Refuerzo Jerárquico (HRL).....	13
2.3. Unity ML-Agents Toolkit.....	13
2.4. Técnicas de Entrenamiento	13
2.4.1. Curriculum Learning.....	13
2.4.2. Reward Shaping	14
2.4.3. Generalization Gap y Transfer Learning	14
2.5. Procesos de Poisson y Simulación de Eventos Discretos	14
2.6. Estado del Arte.....	14
2.6.1. Robótica de Almacenes.....	14
2.6.2. DRL Aplicado a Navegación	15
2.6.3. Sistemas Jerárquicos en Logística	15
3. Metodología	16
3.1. Herramientas de Desarrollo	16
3.1.1. Motor de Simulación: Unity 6	16
3.1.2. Framework de Entrenamiento: Unity ML-Agents.....	17
3.1.3. Entorno Python y Dependencias	17
3.1.4. Visualización de Métricas: TensorBoard.....	18
3.1.5. Lenguaje y Scripts Principales.....	19
3.2. Enfoque de Desarrollo: Iteración Guiada por Fallos	20
3.3. Entorno de Simulación.....	20
3.3.1. Entornos de Entrenamiento (Arenas).....	20
3.3.2. Entorno de Despliegue: Fábrica Industrial	21
3.4. Diseño Experimental.....	23

3.4.1. Variables de Control	24
3.4.2. Variables Experimentales	24
3.4.3. Métricas de Evaluación.....	24
4. Arquitectura del Sistema.....	26
4.1. Visión General: Arquitectura Híbrida Jerárquica	26
4.2. Piloto: Agente de Navegación (Bajo Nivel)	28
4.2.1. Control Cinemático del AGV	28
4.2.2. Espacio de Observaciones.....	30
4.2.3. Espacio de Acciones	31
4.2.4. Sistema de Recompensas	31
4.3. Gestor: Agente de Coordinación (Alto Nivel)	32
4.3.1. Espacio de Observaciones.....	33
4.3.2. Espacio de Acciones Discretas	35
4.3.3. Función de Recompensa	36
4.4. Simulador de Almacén (Capa Lógica).....	37
4.4.1. Generador de Pedidos (GeneradorPedidos.cs).....	38
4.4.2. Lógica del Almacén (GestorAlmacen.cs)	39
4.4.3. Modelo de Datos (Paquete.cs)	39
4.4.4. Cola de Decisiones y Reglas de Reasignación Dinámica	40
4.5. Integración: Puente Físico-Lógico	40
4.5.1. Ciclo de Comunicación Orientado a Eventos	41
4.5.2. Flujo de Ciclo de Vida de un Paquete.....	42
4.5.3. Controlador de Línea Base (FIFO Baseline)	42
4.6. Adaptación del Piloto para Modo Producción	42
5. Experimentos y Resultados.....	44
5.1. Fase E1: Navegación Pura	44
5.1.1. Análisis de Métricas.....	45
5.2. Fase E2: Obstáculos Estáticos	46
5.3. Fase E3: Entornos Procedurales.....	48
5.3.1. Detección del Mínimo Local (Visión de Túnel)	50
5.4. Fase E4: Resiliencia Post-Colisión	51
5.5. Fase E5: Optimización de Trayectorias (Guiado Suave).....	52
5.5.1. Evolución del Sistema de Recompensas (V1 a V5).....	52
5.6. Despliegue en Entorno a Gran Escala.....	53
5.6.1. Detección del Generalization Gap	53
5.6.2. Solución: Normalización Dinámica y Reward Shaping	54
5.6.3. Resultado: Despliegue Exitoso sin Reentrenamiento	55

5.7. Análisis de Comportamientos Emergentes	55
5.8. Análisis Comparativo de Modelos.....	56
5.9. Fase E6: Entrenamiento del Gestor de Alto Nivel.....	57
6. Conclusiones y Trabajo Futuro	62
6.1. Conclusiones.....	62
6.2. Trabajo Futuro	64
Bibliografía	67
Anexo A: Configuraciones de Entrenamiento (YAML).....	71
Anexo B: Glosario de Términos	72

1. Introducción

1.1. Contexto y Motivación

La logística de almacenes se ha convertido en uno de los pilares fundamentales sobre los que se sustenta el comercio moderno. El crecimiento exponencial del comercio electrónico, el cual se vio acelerado a raíz de la pandemia del COVID-19, pone bajo presión la capacidad de los centros de distribución para procesar un incesante volumen de pedidos con tiempos de entrega terriblemente exigentes. Bajo este contexto, la automatización y optimización del transporte interno de mercancías mediante AGV o vehículos de guiado automático se establece como una solución estratégica imprescindible para mejorar la eficiencia, reducir costes operativos y minimizar los posibles errores humanos.

El ejemplo más representativo de este paradigma es la empresa Amazon Robotics, con cientos de miles de unidades operando de forma coordinada en sus centros de distribución a escala global. Sin embargo, la mayoría de las soluciones disponibles en la industria se apoyan en sistemas de navegación rígidos: guías físicas integradas en el suelo —cintas magnéticas, cables inductivos— o rutas preprogramadas que requieren una costosa infraestructura y prediseño que, en la práctica, no tienen capacidad de adaptarse cuando cambia la configuración del almacén.

Frente a esta rigidez, el Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning, DRL; Mnih et al., 2015) propone un enfoque fundamentalmente distinto. En lugar de seguir rutas predefinidas, un agente entrenado mediante DRL aprende a navegar a través de la interacción directa con su entorno, desarrollando políticas de control que pueden generalizar a configuraciones no vistas durante el entrenamiento. Es esta capacidad de adaptación lo que convierte al DRL en un candidato natural para la navegación en almacenes dinámicos, donde la disposición de estanterías, obstáculos o incluso otros vehículos puede variar con frecuencia.

Este Trabajo de Fin de Grado aborda el problema de forma integral separado en dos niveles: desde el entrenamiento de un agente capaz de navegar físicamente por un almacén —el Piloto— hasta el diseño de una capa de gestión logística que coordine las operaciones de recogida y entrega de paquetes —el Gestor—. El entorno de simulación elegido es Unity 6 (Juliani et al., 2018), con ML-Agents (Unity Technologies, 2024) como framework de

entrenamiento, una combinación que permite integrar un entorno físico realista con las herramientas actuales de Reinforcement Learning.

El código fuente completo del sistema está disponible en el repositorio público del proyecto: <https://github.com/daniel27bc/Warehouse-AGV-Deep-RL>. Las clases y métodos que se citan a lo largo de este y el resto de capítulos pueden consultarse íntegramente en dicho repositorio.

1.2. Objetivos del Trabajo

El objetivo central de este trabajo es diseñar, implementar y evaluar un sistema autónomo de navegación y gestión logística para un almacén simulado, organizado en una arquitectura jerárquica que combina Aprendizaje por Refuerzo Profundo con simulación de eventos discretos. Esta meta se concreta en los siguientes objetivos específicos:

Objetivo 1. Desarrollo del agente de navegación (Piloto): diseñar y entrenar un agente de bajo nivel que conduzca un AGV simulado desde cualquier posición hasta una coordenada de destino arbitraria, esquivando obstáculos de forma autónoma mediante el algoritmo PPO (Proximal Policy Optimization), un algoritmo de gradiente de política de referencia en el campo (Schulman et al., 2017), con espacio de acción continuo.

Objetivo 2. Diseño de un aprendizaje curricular progresivo (*curriculum learning*): estructurar el entrenamiento en fases de complejidad creciente —desde sala vacía hasta entornos procedurales— que permitan al agente adquirir capacidades de forma incremental sin degradar las habilidades ya aprendidas.

Objetivo 3. Análisis y documentación de la brecha de generalización (*Generalization Gap*): investigar, diagnosticar y resolver el fenómeno de degradación del rendimiento al transferir el agente desde entornos de entrenamiento controlados a un escenario a gran escala.

Objetivo 4. Implementación del simulador logístico (Gestor): construir un simulador de eventos discretos que modele el flujo de paquetes en el almacén, incluyendo la generación estocástica de pedidos mediante proceso de Poisson y la gestión del ciclo de vida de cada paquete.

Objetivo 5. Integración de la arquitectura jerárquica completa: conectar las capas de navegación y gestión logística en un sistema funcional *end-to-end*, validando que el AGV ejecuta correctamente misiones de recogida y entrega en la fábrica simulada.

Objetivo 6. Validación en entorno realista: desplegar y evaluar el sistema completo en un escenario industrial de alta fidelidad (80×80 metros), demostrando la viabilidad del enfoque en condiciones próximas a las de un centro de distribución real.

1.3. Alcance y Limitaciones

El alcance de este trabajo abarca el diseño, la implementación, el entrenamiento y la validación cuantitativa de una arquitectura híbrida de aprendizaje por refuerzo jerárquico (HRL; Pateria et al., 2021) para la optimización del flujo de materiales en un almacén. Esto comprende la creación de entornos de simulación física en 3D en Unity 6, el entrenamiento de un agente piloto de bajo nivel mediante PPO para navegación continua, la implementación de un agente gestor de alto nivel para asignación discreta de tareas con reasignación dinámica, y la validación cuantitativa contra heurísticas industriales (por ejemplo, FIFO (*First In, First Out*) y *Greedy*).

En cuanto a las limitaciones, el estudio se acota al ámbito de la simulación virtual, asumiendo lecturas de sensores ideales y transmisiones de datos sin latencia. Adicionalmente, debido a la complejidad geométrica y los límites de convergencia temporal en el espacio de observaciones, el modelo final del Gestor de Alto Nivel está dimensionado para una flota activa de $N = 2$ robots con un vector de estado de tamaño fijo, postergándose la generalización a flotas dinámicas de tamaño variable (mediante Buffer Sensors y capas de atención) para trabajos futuros en la sección correspondiente.

Dicho esto, existen tres limitaciones relevantes que deben tenerse en cuenta:

- **Gestor basado en reglas.** La capa logística emplea actualmente una estrategia FIFO. La sustitución de esta heurística por un segundo agente de RL entrenado para optimizar la asignación es la siguiente fase prevista.
- **Entorno estático.** Los obstáculos del escenario final —columnas, maquinaria, muros— son fijos. El entrenamiento con obstáculos dinámicos, como otros vehículos u operarios en movimiento, se identifica como línea de trabajo futuro.

- **Simulación exclusiva.** Todo el desarrollo y la validación se realizan en entorno simulado. La transferencia a hardware real (*sim-to-real transfer*) queda fuera del alcance de este TFG, aunque el diseño incorpora restricciones físicas realistas — masa, velocidad, inercia— para facilitar una futura transferencia.

2. Marco Teórico y Estado del Arte

Este capítulo presenta los fundamentos teóricos sobre los que se sustenta el desarrollo del proyecto. Se describen los conceptos esenciales del Aprendizaje por Refuerzo, el algoritmo de optimización empleado (PPO), el toolkit de entrenamiento utilizado (Unity ML-Agents), la arquitectura jerárquica propuesta, las técnicas de entrenamiento aplicadas y el modelado matemático del sistema logístico. Finalmente, se revisa el estado del arte en robótica de almacenes y navegación autónoma con RL.

Dado el carácter técnico del trabajo, conviene señalar que los acrónimos y términos especializados que aparecen a lo largo de la memoria se recogen y explican en el Anexo B (Glosario de Términos).

2.1. Aprendizaje por Refuerzo

El Aprendizaje por Refuerzo (Reinforcement Learning, RL) es un área del aprendizaje automático en el que un agente aprende a tomar decisiones óptimas mediante la interacción directa con un entorno. A diferencia del aprendizaje supervisado, donde se dispone de un conjunto de datos etiquetados, en RL el agente debe descubrir qué acciones maximizan una señal de recompensa acumulada a largo plazo a través de un proceso de prueba y error (Sutton y Barto, 2018).

El RL se distingue por tres características fundamentales: la ausencia de un supervisor explícito que indique la acción correcta, la naturaleza secuencial de las decisiones (donde cada acción afecta a los estados futuros) y el dilema entre exploración de nuevas estrategias y explotación de las ya conocidas.

2.1.1. Proceso de Decisión de Markov (MDP)

La formalización matemática estándar del RL se realiza mediante un Proceso de Decisión de Markov (MDP; Sutton y Barto, 2018), definido como (S, A, P, R, γ) , donde S representa el espacio de estados, A es el espacio de acciones, P es la función de transición, R es la función de recompensa, y $\gamma \in [0,1]$ es el factor de descuento que pondera la importancia relativa de las recompensas futuras frente a las inmediatas.

La propiedad de Markov establece que el estado futuro depende únicamente del estado presente y la acción tomada, no del historial completo de estados anteriores. Esta propiedad

simplifica enormemente el problema de decisión y permite el uso de programación dinámica y métodos iterativos para encontrar soluciones óptimas.

Política (π): un mapeo de estados a acciones que define el comportamiento del agente. Puede ser determinista ($\pi(s) = a$) o estocástica ($\pi(a | s) = P(a_t = a | s_t = s)$). El objetivo del RL es encontrar la política óptima π^* que maximice la recompensa acumulada esperada.

Función de Valor $V\pi(s)$: representa la recompensa acumulada esperada partiendo del estado s y siguiendo la política π . Formalmente, $V_\pi(s) = \mathbb{E}_\pi[\sum_t \gamma^t R_t | s_0 = s]$.

Función de Acción-Valor $Q\pi(s,a)$: representa la recompensa esperada al tomar la acción a en el estado s y posteriormente seguir la política π .

Función de Ventaja $A\pi(s,a)$: definida como $A\pi(s,a) = Q\pi(s,a) - V\pi(s)$, cuantifica cuánto mejor es una acción específica respecto al promedio de acciones en ese estado. Esta función es central en el algoritmo PPO.

2.1.2. Métodos Policy Gradient

Los métodos de gradiente de política (Policy Gradient) optimizan directamente los parámetros θ de una política parametrizada π_θ mediante ascenso por gradiente. El teorema del gradiente de política (Sutton et al., 2000) establece que el gradiente del objetivo puede estimarse como: $\nabla J(\theta) = \mathbb{E}_\pi[\nabla \log \pi_\theta(a | s) A_\pi(s, a)]$.

Las ventajas principales incluyen: su capacidad natural para manejar espacios de acción continuos, la posibilidad de aprender políticas estocásticas que facilitan la exploración y su estabilidad en entornos con alta dimensionalidad.

2.1.3. PPO (Proximal Policy Optimization)

Proximal Policy Optimization (Schulman et al., 2017) es un algoritmo de la familia Policy Gradient diseñado para ofrecer un equilibrio óptimo entre estabilidad, eficiencia y simplicidad. PPO resuelve la sensibilidad al tamaño del paso de actualización mediante un mecanismo de recorte (clipping) más simple que la optimización restringida de su predecesor TRPO.

Arquitectura Actor-Critic. PPO emplea dos redes neuronales: el Actor, que parametriza la política $\pi_\theta(a | s)$ y decide qué acción tomar, y el Critic, que estima la función de valor $V_\phi(s)$ como línea base para reducir la varianza del gradiente.

Función Objetivo con Recorte. $L(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)A_t)]$, donde $r_t(\theta)$ es el ratio de probabilidad entre política nueva y antigua, y ε (típicamente 0.2) limita la magnitud de las actualizaciones.

GAE (Generalized Advantage Estimation). Para estimar la ventaja A_t , PPO utiliza GAE (Schulman et al., 2016), que ofrece un compromiso controlable entre sesgo y varianza mediante el parámetro λ .

En este proyecto, PPO se aplica al Piloto con un espacio de acción continuo bidimensional. Su elección frente a SAC se justifica por su estabilidad demostrada en navegación, su integración nativa en ML-Agents y su menor sensibilidad a hiperparámetros.

2.1.4. Consideración de DQN frente a PPO para el Control Discreto

En problemas de toma de decisiones discretas y de baja dimensionalidad como el que afronta el Gestor de Alto Nivel, los algoritmos basados en funciones de valor, como DQN (Deep Q-Network) (Mnih et al., 2015), resultan teóricamente idóneos. DQN estima de forma directa el valor esperado de cada acción para un estado dado mediante la ecuación de Bellman, lo que suele ofrecer una mayor eficiencia de muestreo en espacios de acción finitos.

Sin embargo, el *pipeline* estándar de entrenamiento en Unity ML-Agents limita los algoritmos nativos disponibles en su interfaz de línea de comandos (mlagents-learn) a PPO y SAC. La implementación práctica de DQN habría exigido conectar el simulador de Unity con frameworks externos de Python (como Gymnasium y Stable-Baselines3) mediante la API de bajo nivel, incrementando drásticamente la sobrecarga de comunicación por sockets y la complejidad del entorno de desarrollo. Por ello, se optó por emplear PPO, el cual mapea las predicciones numéricas de la red hacia una distribución de probabilidad para seleccionar las acciones discretas. PPO ofrece una buena estabilidad de entrenamiento y permite mantener la coherencia metodológica y de infraestructura con el agente Piloto de bajo nivel.

2.2. Aprendizaje por Refuerzo Jerárquico (HRL)

El HRL aborda la dificultad de resolver tareas complejas con un único nivel de decisión. Descompone la tarea en múltiples niveles de abstracción, donde cada capa opera a una frecuencia temporal y nivel de detalle diferentes (Barto y Mahadevan, 2003).

Options Framework (Sutton, Precup y Singh, 1999). Formaliza las abstracciones temporales como “opciones”: políticas temporalmente extendidas con condición de inicio, política interna y condición de terminación.

Feudal Networks (Vezhnevets et al., 2017). Propone un “manager” que fija sub-objetivos y un “worker” que los ejecuta. Inspiración directa para este TFG y su diseño Gestor-Piloto.

En nuestro proyecto, la necesidad de HRL emergió naturalmente del análisis del problema de mínimos locales detectado experimentalmente (Sección 5.3), justificando la incorporación de una capa superior con visión global.

2.3. Unity ML-Agents Toolkit

Unity ML-Agents (Juliani et al., 2018) es un toolkit de código abierto que permite crear, entrenar y desplegar agentes inteligentes en entornos 3D. Actúa como puente entre Unity (C#) y PyTorch (Python) mediante protocolo gRPC.

Clase Agent. Componente central en Unity que implementa `OnEpisodeBegin()`, `CollectObservations()` y `OnActionReceived()`.

Sensores. `VectorSensor` para observaciones numéricas y `RayPerceptionSensor3D` que simula un LiDAR mediante rayos configurables.

Exportación. Los modelos se exportan en formato ONNX, ejecutable directamente en Unity sin Python en producción.

2.4. Técnicas de Entrenamiento

2.4.1. Curriculum Learning

Estrategia inspirada en la pedagogía humana (Bengio et al., 2009): basada en la presentación progresiva de versiones cada vez más complejas de un problema. En este proyecto se estructura en fases E1–E4, cada una heredando los pesos de la anterior.

2.4.2. Reward Shaping

El diseño de la función de recompensa determina la velocidad de convergencia y la calidad de la política aprendida. En entornos con recompensas escasas (sparse), el agente solo recibe feedback al completar con éxito la tarea completa, lo que ralentiza el aprendizaje en trayectos largos. El Reward Shaping (Ng et al., 1999) mitiga este problema añadiendo una función de potencial $\Phi(s)$ al feedback inmediato: $R'(s, a, s') = R(s, a, s') + \gamma\Phi(s') - \Phi(s)$. Esta formulación matemática garantiza que la adición de premios auxiliares no altere la política óptima. En este proyecto, el Reward Shaping ha sido crítico para guiar al Piloto a través de trayectos largos y evitar atascos locales.

2.4.3. Generalization Gap y Transfer Learning

La brecha de generalización (Cobbe et al., 2019) designa la degradación al operar en condiciones diferentes a las del entrenamiento. El OOD Shift ocurre cuando las observaciones caen fuera de la distribución estadística de entrenamiento, saturando las funciones de activación. Las técnicas de mitigación incluyen normalización de observaciones, aleatorización de dominio (domain randomization) y ajuste fino (fine-tuning).

2.5. Procesos de Poisson y Simulación de Eventos Discretos

El proceso de Poisson modela la ocurrencia de eventos aleatorios independientes a tasa media λ . El tiempo entre llegadas sigue una distribución exponencial $f(x) = \lambda e^{-\lambda x}$. La generación computacional se realiza mediante transformada inversa: $X = -\ln(U)/\lambda$ con $U \sim \text{Uniform}(0,1)$. El proyecto emplea además un simulador de eventos discretos (DES) para gestionar el estado lógico del almacén independientemente de las físicas de Unity (Law, 2015).

2.6. Estado del Arte

2.6.1. Robótica de Almacenes

Amazon Robotics opera cientos de miles de robots bajo coordinación centralizada. Otras empresas como Ocado robotics despliega enjambres sobre rejillas elevadas. Los AGV tradicionales se basan en guías fijas, careciendo de capacidad de adaptación. La propuesta de este TFG emplea navegación aprendida mediante RL, capaz de generalizar a configuraciones arbitrarias.

2.6.2. DRL Aplicado a Navegación

Tai et al. (2017) demostraron la viabilidad de navegación sin mapa mediante RL continuo. Tobin et al. (2017) introdujeron la aleatorización de dominio para cerrar el gap sim-to-real. Juliani et al. (2018) validaron ML-Agents como plataforma para IA embodied.

2.6.3. Sistemas Jerárquicos en Logística

Pateria et al. (2021) analizan de manera exhaustiva el Aprendizaje por Refuerzo Jerárquico (HRL), destacando métodos de descomposición temporal como la arquitectura de opciones o la delegación de objetivos lógicos. Por otro lado, trabajos como el de Nazari et al. (2018) proponen el uso de DRL para resolver el problema de enrutamiento de vehículos (VRP) bajo demanda estocástica. El presente proyecto se sitúa en la convergencia de ambas áreas: propone una arquitectura jerárquica de dos niveles en la que un planificador lógico (Gestor) asigna de forma discreta los destinos y un Piloto de bajo nivel ejecuta el guiado cinemático continuo en el almacén, incorporando la capacidad dinámica de reasignación dinámica.

3. Metodología

Este capítulo describe las herramientas empleadas, el enfoque de desarrollo adoptado, el diseño experimental que estructura las fases de entrenamiento y el entorno de simulación utilizado. La metodología seguida se caracteriza por un ciclo iterativo de diseño, entrenamiento, evaluación y refinamiento, donde cada fase de experimentación se construye sobre los resultados y las limitaciones detectadas en la anterior.

3.1. Herramientas de Desarrollo

3.1.1. Motor de Simulación: Unity 6

El entorno de simulación se ha desarrollado íntegramente en Unity 6 (versión 6000.0.40f1), un motor de desarrollo 3D que ofrece un sistema de físicas basado en NVIDIA PhysX, un *pipeline* de renderizado configurable (Universal Render Pipeline, URP) y un ecosistema maduro de assets reutilizables (recursos como modelos 3D, texturas o scripts). La elección de Unity frente a alternativas como Unreal Engine, Gazebo o PyBullet se justifica por tres factores: su integración nativa con el toolkit ML-Agents, la disponibilidad de assets industriales de alta calidad visual para el escenario del almacén, la posibilidad de desarrollar toda la lógica del sistema en C# dentro del mismo entorno y su sencillez de uso frente a otras alternativas gracias a una previa familiaridad con el software. El entorno de trabajo resultante puede verse en la Figura 3.1, con el escenario de la fábrica industrial ya cargado en el editor.

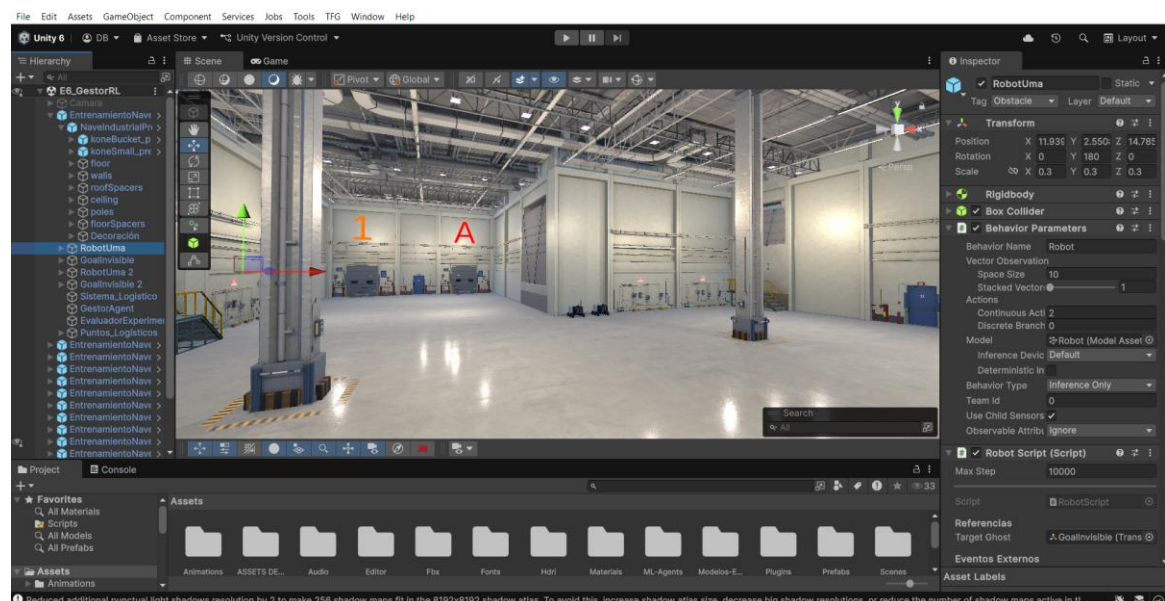


Figura 3.1: Interfaz del editor de Unity con el escenario de la fábrica industrial cargado

Este motor permite la creación de las denominadas "escenas", las cuales actúan como contenedores cerrados de física, geometría y lógica de scripts. Para este proyecto, el uso de escenas independientes resultó fundamental, ya que posibilitó la creación de múltiples arenas de entrenamiento en paralelo para acelerar la recopilación de datos, así como el aislamiento de fases experimentales en versiones previas sin peligro de sobrescribir el mapa industrial final. Esto agiliza la toma y comparación de datos de entrenamiento en TensorBoard (Abadi et al., 2016), una interfaz para visualizar los resultados que se explicará a continuación.

3.1.2. Framework de Entrenamiento: Unity ML-Agents

El entrenamiento de las redes neuronales se realiza mediante Unity ML-Agents, un paquete de assets de código abierto proporcionado por la empresa para este tipo de investigaciones. ML-Agents se encarga de establecer una comunicación bidireccional entre el entorno Unity (C#) y el trainer de Python (PyTorch) a través de protocolo gRPC.

El flujo de trabajo típico consiste en: definir el comportamiento del agente en C# (observaciones, acciones, recompensas), también es posible configurar los parámetros del algoritmo en un archivo YAML, lanzar el entrenamiento desde línea de comandos y monitorizar la convergencia en tiempo real.

3.1.3. Entorno Python y Dependencias

El lado de Python se gestiona mediante un entorno Conda dedicado (llamado mlagents) con Python 3.10 y PyTorch como backend de computación. El entrenamiento puede ejecutarse tanto en CPU como en GPU (CUDA), aunque dada la dimensionalidad relativamente baja del espacio de observaciones (10 floats + sensores de rayo), la diferencia de rendimiento entre ambos modos es marginal en este proyecto.

3.1.4. Visualización de Métricas: TensorBoard

TensorBoard es una herramienta que se utiliza para el análisis y visualización de las métricas de entrenamiento. Permite monitorizar en tiempo real la recompensa acumulada por episodio, la longitud media de los episodios, las pérdidas de la política y del valor, y la entropía de la distribución de acciones. La interpretación correcta de estas curvas, incluyendo el uso adecuado del parámetro de suavizado (smoothing), que filtra el ruido de la señal para revelar la tendencia, resulta esencial para diagnosticar problemas de entrenamiento y tomar decisiones informadas sobre cuándo detener o modificar el proceso. La interfaz de TensorBoard en la que se visualiza la evolución de estas métricas aparece en la Figura 3.2.

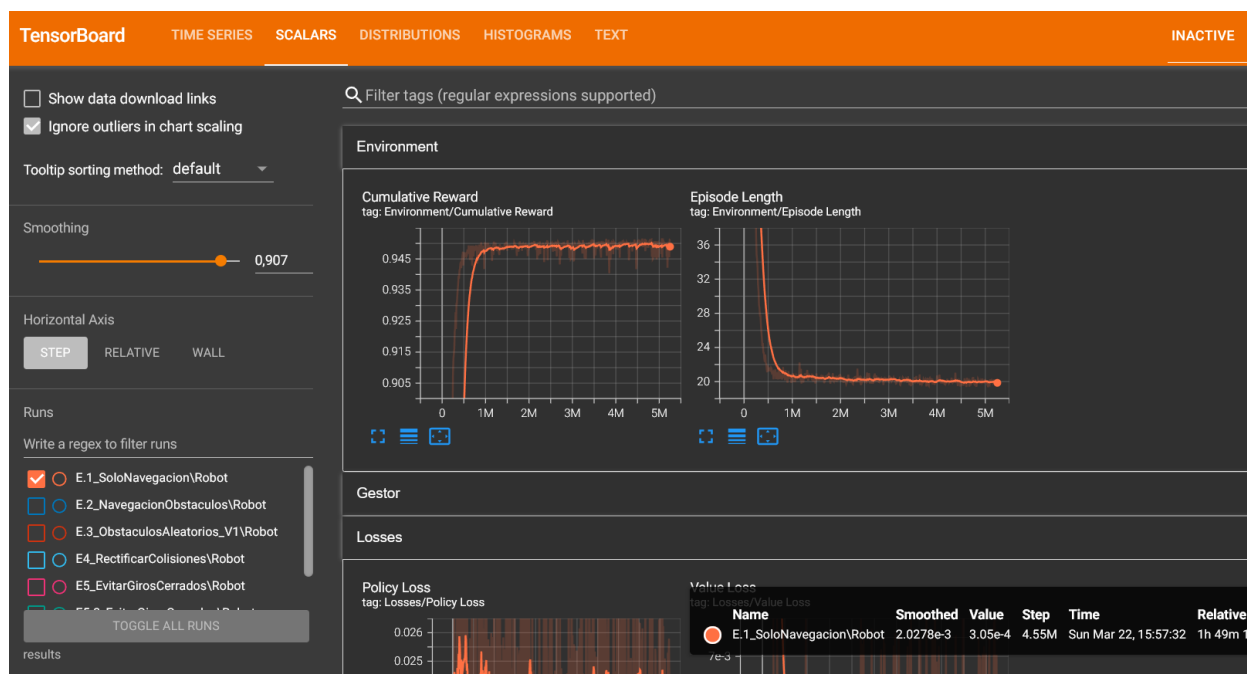


Figura 3.2: Interfaz de TensorBoard mostrando la evolución de las métricas de entrenamiento del agente.

3.1.5. Lenguaje y Scripts Principales

Toda la lógica del sistema se ha implementado en C# dentro de Unity. La Tabla 3.1 resume los scripts principales del proyecto y su responsabilidad funcional. Todos los códigos son de acceso público en el [repositorio](#).

Script / Componente	Ubicación en Proyecto	Responsabilidad Funcional	Capa del Sistema
RobotScript.cs	Assets/Scripts/	Control cinemático, lectura de sensores (LiDAR + vectores) y guiado físico del robot.	Piloto (Bajo Nivel)
GestorAlmacen.cs	Assets/Scripts/Almacen/	Lógica del Almacén (DES): colas lógicas, tiempos de espera y estado de robots.	Simulador (Lógica del Almacén)
GestorAgent.cs	Assets/Scripts/Almacen/	Agente decisor DRL (PPO). Procesa las 38 observaciones y emite las 3 acciones discretas.	Gestor (Decisor Alto Nivel)
GeneradorPedidos.cs	Assets/Scripts/Almacen/	Generación estocástica de paquetes siguiendo una distribución de Poisson parametrizable.	Simulador (Generador de Pedidos)
EvaluadorExperimentos.cs	Assets/Scripts/Almacen/	Automatización del protocolo comparativo de 30 runs y registro en resultados_E6.csv.	Evaluación / Métricas

Tabla 3.1: Listado de los principales scripts y sus funciones.

3.2. Enfoque de Desarrollo: Iteración Guiada por Fallos

El desarrollo del proyecto sigue un enfoque iterativo que puede describirse como “curriculum learning emergente”: en lugar de diseñar a priori un plan rígido de fases, cada nueva fase de entrenamiento surge como respuesta directa a una limitación o fallo detectado en la fase anterior. Este enfoque, aunque no planificado inicialmente, resultó ser la metodología natural para un proyecto de investigación aplicada donde el comportamiento del agente es inherentemente impredecible. El ciclo de trabajo en cada fase sigue un patrón consistente de cinco etapas:

- 1. Identificación del problema.** Se detecta una limitación en el comportamiento del agente, ya sea mediante observación visual de las inferencias, análisis de las métricas de TensorBoard o pruebas en entornos no vistos durante el entrenamiento. Por ejemplo, en la Fase E3 se detectó que el robot quedaba atrapado en rendijas estrechas (mínimo local).
- 2. Análisis teórico.** Se investiga la causa subyacente del fallo desde una perspectiva teórica. En el ejemplo anterior, se determinó que el fenómeno es un comportamiento matemáticamente correcto de una política puramente local sin visión global.
- 3. Diseño de la solución.** Se modifica el entorno, la función de recompensa o la arquitectura del sistema para abordar el problema. Los cambios se diseñan para ser mínimos e incrementales, preservando las capacidades adquiridas en fases previas.
- 4. Entrenamiento y evaluación.** Se ejecuta el nuevo entrenamiento (heredando pesos cuando es posible) y se analizan las métricas resultantes. Los criterios de éxito se definen cuantitativamente: recompensa media superior a un umbral, longitud de episodio estable, y value loss decreciente.
- 5. Validación en contexto.** Se despliega el modelo en el entorno objetivo (la fábrica) para verificar que la mejora se traslada al escenario real de uso. Si no es así, se reinicia el ciclo con un nuevo diagnóstico.

3.3. Entorno de Simulación

3.3.1. Entornos de Entrenamiento (Arenas).

Las fases E1 a E5 utilizan arenas de entrenamiento modulares de 8×8 metros, diseñadas como escenarios controlados donde se aísla una capacidad específica del agente. Cada arena se almacena como un Prefab (plantilla reutilizable de un objeto o conjunto de objetos

que conserva todos sus componentes y configuración) independiente en Unity, lo que permite restaurar cualquier fase anterior a voluntad. La Figura 3.3 compara las tres arenas resultantes E1 vacía, E2 con obstáculos fijos y E3 con obstáculos aleatorios.

Comparativa de las Tres Arenas de Entrenamiento del Piloto

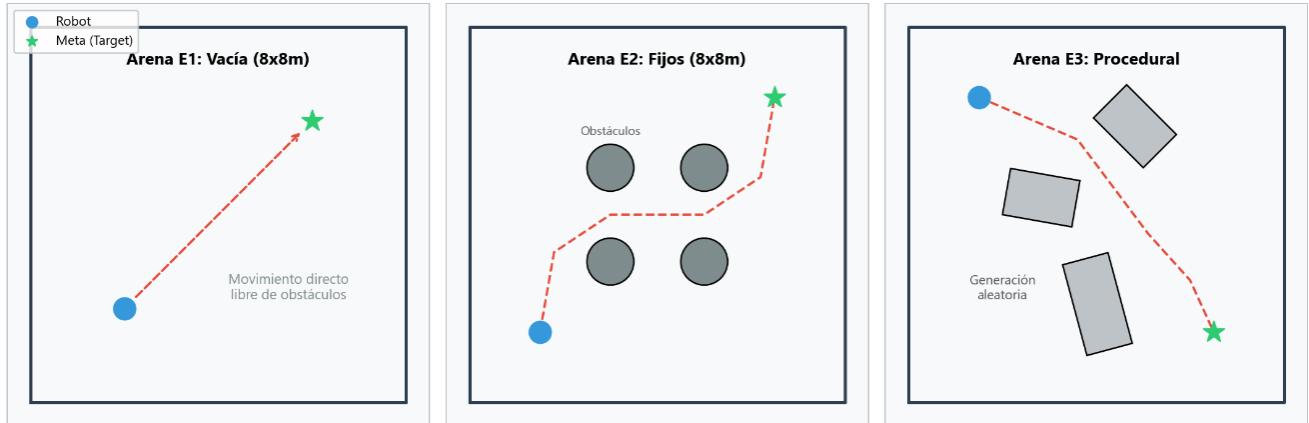


Figura 3.3: Comparativa visual de las tres arenas: E1 (completamente vacía), E2 (obstáculos fijos) y E3 (aleatorios).

Además ML-Agents permite instanciar múltiples copias de estas arenas simultáneamente para acelerar la recolección de experiencias durante el entrenamiento (ver Figura 3.4).

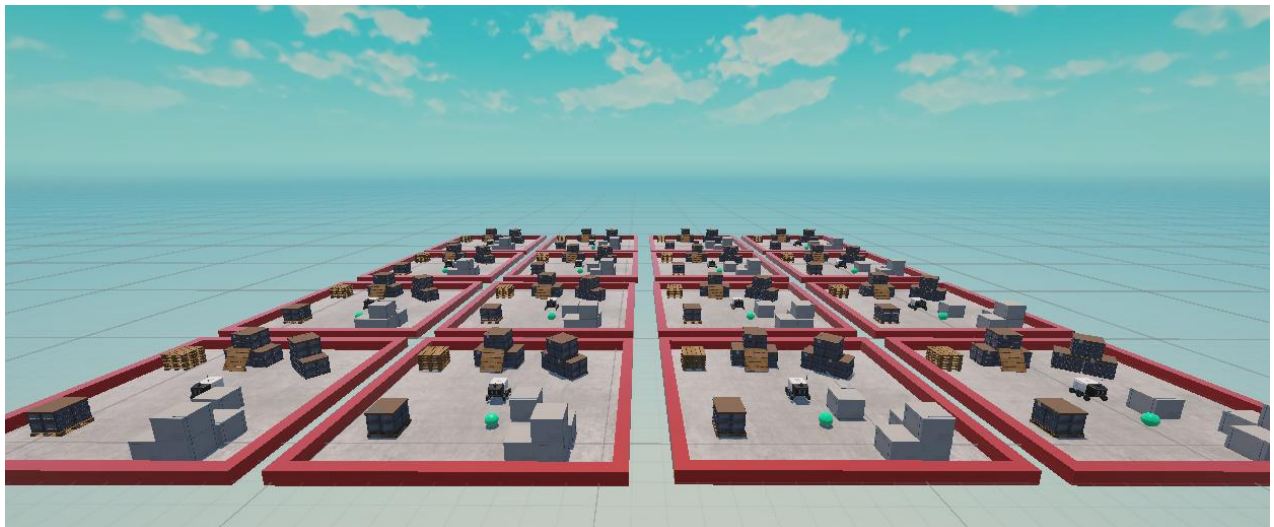


Figura 3.4: Ejemplo de múltiples instancias (16) Para el E4 en paralelo.

3.3.2. Entorno de Despliegue: Fábrica Industrial

Para la validación final del sistema, se migró el proyecto a un nuevo entorno, para ello se adaptó un asset público y gratuito ofrecido por Unity en su tienda de assets. Tras modificar la escena para adaptarla a los requisitos del problema, se consiguió un escenario realista y visualmente fiel a un centro de distribución logístico. El mapa incluye suelos reflectantes industriales, columnas estructurales, muelles de carga, maquinaria y luminarias de techo.

Las dimensiones aproximadas del espacio navegable son de 80 x 80 metros, lo que supone un incremento de escala de 10× respecto a las arenas de entrenamiento. La Figura 3.5 ofrece una vista general de este entorno final.

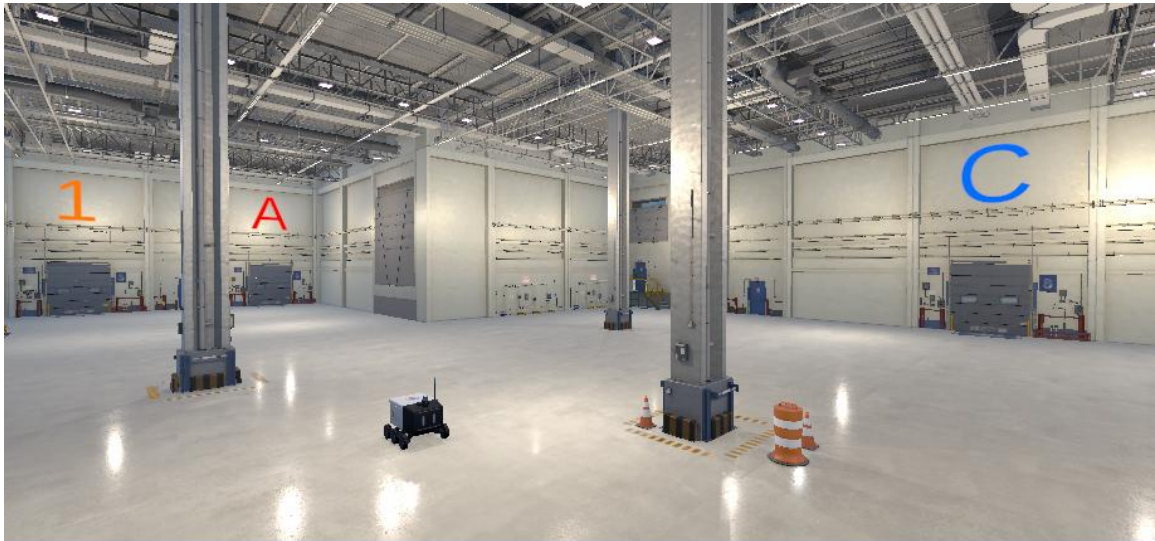


Figura 3.5: Vista general del entorno final de la fábrica industrial.

El mapa de la fábrica (Figura 3.6) se ha configurado con los siguientes puntos de control logísticos:

- **Puntos de entrada (2):** muelles de carga señalizados en color naranja con los números 1 y 2, donde llega la mercancía al sistema.
- **Puntos de salida (3):** zonas de empaquetado identificadas con las letras A, B y C, donde se entregan los pedidos procesados.



Figura 3.6: Vista aérea del mapa de la fábrica, zona de circulación y ubicación de puntos de control (Entradas 1 y 2, Salidas A, B y C).

3.4. Diseño Experimental

El proceso de entrenamiento se estructura en seis fases experimentales (E1–E6), cada una con un objetivo específico, una configuración de entorno particular y unos criterios de éxito medibles. La Tabla 3.2 presenta un resumen de todas las fases.

Fase / Modelo	Objetivo de la Fase	Entorno / Arena	Pasos Totales	Resultado Clave / Capacidad
E1 (Básico)	Validar navegación básica en entorno abierto.	Arena vacía (8×8 m).	5.25 Millones	Movimiento directo y fluido hacia la meta sin desvíos.
E2 (Estático)	Evitación de obstáculos fijos.	Arena con 4 columnas fijas (8×8 m).	15.0 Millones	Esquiva obstáculos estáticos en línea visual directa.
E3 (Procedural)	Generalización a mapas geométricos dinámicos.	Arena con bloques aleatorios (8×8 m).	38.7 Millones (acumulados)	99% de éxito en mapas y configuraciones no vistos.
E4 (Resiliente)	Estabilidad ante roces y colisiones leves.	Arena con bloques aleatorios (8×8 m).	61.5 Millones (acumulados)	Desatasco autónomo y resiliencia ante colisiones.
E5 (Guiado Suave)	Suavizado de trayectos y control de deriva.	Pasillos de fábrica (80×80 m).	80.0 Millones (acumulados)	Trayectorias estables y lineales sin oscilaciones (zigzag).
E6 (Coordinador)	Integración del sistema y asignación de tareas.	Fábrica completa (80×80 m), N=2 AGVs.	2.0 Millones (Gestor)	Reducción del tiempo medio de permanencia en sistema bajo carga alta (32.78%).

Tabla 3.2: Resumen de entrenamientos y resultados.

3.4.1. Variables de Control

Para garantizar la validez de los resultados y la comparabilidad entre fases, se mantienen constantes las siguientes variables a lo largo de todo el proyecto:

- **Arquitectura de la red neuronal:** la estructura de la red (número de capas ocultas, unidades por capa) y el espacio de observaciones (Space Size = 10 + RayPerceptionSensor3D) permanecen invariantes desde la Fase E1. Esto es un requisito técnico de ML-Agents: modificar la arquitectura de entrada invalida los pesos aprendidos.
- **Restricciones físicas del AGV:** la masa (25 kg), velocidad máxima lineal (1.5 m/s), velocidad máxima angular (200°/s) y la restricción de marcha atrás se mantienen idénticas en todas las fases.
- **Algoritmo de entrenamiento:** PPO con los mismos hiperparámetros base.

3.4.2. Variables Experimentales

Las variables que se modifican entre fases son:

- **Complejidad del entorno:** desde sala vacía (E1) hasta fábrica industrial completa (E6), pasando por obstáculos fijos (E2), procedurales (E3) y la transición de escala.
- **Función de recompensa:** evoluciona progresivamente: recompensa sparse con muerte instantánea (E1–E3), colisión no letal con castigo continuo (E4), y reward shaping por delta de distancia con normalización dinámica (E5).
- **Generación procedimental:** a partir de E3, el entorno cambia cada episodio. La distribución estadística de los obstáculos (gaussiana centrada) y la validación de pasillos transitables son parámetros específicos de estas fases.

3.4.3. Métricas de Evaluación

El rendimiento de cada fase se evalúa mediante cuatro métricas principales proporcionadas por TensorBoard:

Cumulative Reward: recompensa total acumulada por episodio. El valor máximo teórico depende de la función de recompensa de cada fase. Un valor estable y alto indica convergencia.

Episode Length: número medio de pasos (steps) por episodio. Valores bajos y estables indican que el agente resuelve la tarea con rapidez y eficiencia.

Value Loss: error de predicción del Critic. Un descenso sostenido indica que la red aprende a predecir correctamente las recompensas futuras del entorno.

Policy Loss: magnitud de cambio en la política. En PPO, las oscilaciones son normales y esperadas debido al mecanismo de clipping. Una divergencia monótona indicaría inestabilidad.

La Figura 3.7 reúne un ejemplo de estas cuatro métricas principales tal como las presenta TensorBoard.

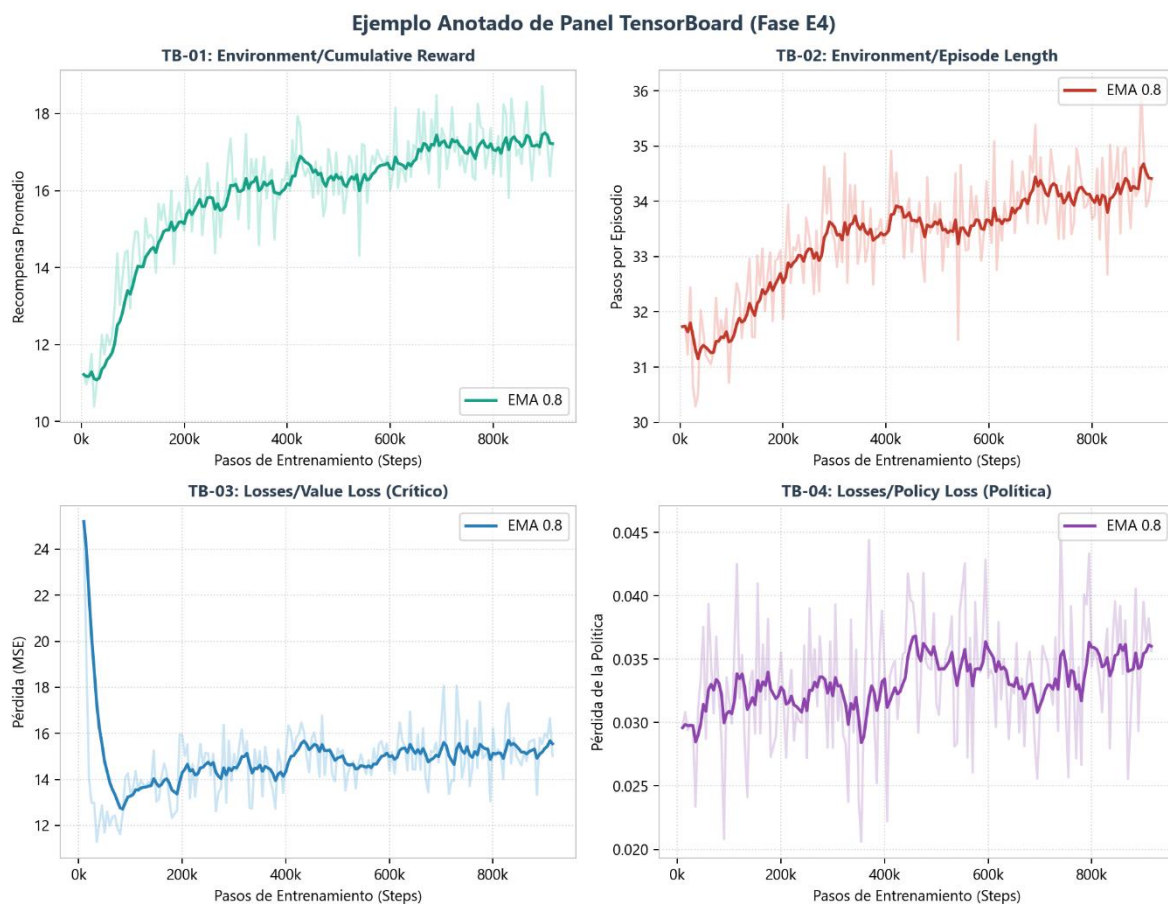


Figura 3.7: Ejemplo de resultados de TensorBoard con las 4 métricas principales del entrenamiento PPO (Recompensa Acumulada, Longitud del Episodio, Pérdida del Crítico y Pérdida de la Política).

4. Arquitectura del Sistema

Este capítulo describe en detalle la arquitectura completa del sistema desarrollado. El diseño final sigue un paradigma jerárquico de dos niveles inspirado en los principios del Aprendizaje por Refuerzo Jerárquico (HRL): una capa de bajo nivel encargada de la navegación física reactiva del AGV (el Piloto), y una capa de alto nivel responsable de la gestión logística global del almacén (el Gestor). Ambas capas operan de forma desacoplada, comunicándose exclusivamente a través de coordenadas de destino, lo que permite entrenar, evaluar y sustituir cada componente de manera independiente.

4.1. Visión General: Arquitectura Híbrida Jerárquica

La arquitectura del sistema se organiza estructuralmente en **dos niveles de agentes inteligentes** (el Nivel Superior y el Nivel Inferior) que se comunican, interactúan y se sincronizan a través de una **Capa Intermedia de Coordinación y Simulación**. Esta separación permite que cada nivel de decisión y aprendizaje opere en su propia escala temporal y espacial, desacoplando la física del movimiento del control logístico abstracto. Esta arquitectura jerárquica completa se sintetiza en la Figura 4.1. Y se detalla en profundidad a lo largo del desarrollo del capítulo en sus diferentes entradas.

Arquitectura Jerárquica e Integración del Sistema HRL



Figura 4.1: Arquitectura jerárquica completa

1. Nivel Superior: Decisión, Planificación e Inteligencia (El Gestor)

En lo alto de la jerarquía opera el **Agente Gestor** (implementado en `GestorAgent.cs`), que constituye el núcleo de decisión de alto nivel del sistema. Se trata de un agente entrenado mediante Aprendizaje por Refuerzo Profundo que implementa una política PPO discreta.

- **Función:** tomar decisiones discretas de asignación y reasignación dinámica de tareas (por ejemplo, decidir a qué puerta de entrada enviar a un robot que solicita trabajo).
- **Abstracción:** el Gestor no interactúa de forma directa con la física de los robots ni controla sus motores. Toma sus decisiones basándose en la información abstracta que le proporciona la capa intermedia y ejecuta sus acciones simplemente reposicionando las coordenadas del objetivo tridimensional del robot (`TargetGhost`).
-

2. Capa Intermedia de Coordinación y Simulación

Entre ambos niveles operacionales se sitúa la **Capa de Coordinación y Simulación**. Esta capa no contiene ningún agente de aprendizaje por refuerzo; es una infraestructura de software tradicional escrita en C# cuya única función es servir de puente lógico-físico y simular el entorno operacional del almacén. Se compone de dos scripts principales:

- **Generador de Pedidos (`GeneradorPedidos.cs`):** simula de forma estocástica la demanda mediante un **proceso de Poisson** (generación de tiempos exponenciales), determinando aleatoriamente la puerta de origen y el muelle de salida de cada caja.
- **Lógica del Almacén (`GestorAlmacen.cs`):** mantiene el estado operativo en tiempo real. Gestiona las colas físicas, controla y registra los tiempos de espera de los paquetes, monitoriza las posiciones de los robots y gestiona las observaciones físicas (como la detección de llegadas y los tiempos límite por entrega).

Esta capa traduce las decisiones abstractas del Nivel Superior en cambios físicos en la escena de Unity y asocia visualmente los paquetes a los AGVs en el momento adecuado.

3. Nivel Inferior: Control Cinemático y Navegación Reactiva (El Piloto)

En la base del sistema opera el **Agente Piloto** (implementado en RobotScript.cs), que representa el control táctico del AGV físico. Es un agente entrenado de manera independiente mediante PPO continuo.

- **Función:** ingiere observaciones locales y continuas (distancias, ángulos y lecturas del sensor LiDAR) para emitir comandos directos sobre la aceleración lineal y velocidad angular del vehículo. Su meta es alcanzar físicamente las coordenadas del *TargetGhost* que el nivel superior ha posicionado para él.
- **Aislamiento:** el Piloto opera de forma continua en el tiempo físico de la simulación. Desconoce por completo los objetivos logísticos de su viaje, el inventario del almacén, el estado de las colas o las decisiones del Gestor; su única función es navegar de manera óptima hacia la coordenada asignada esquivando obstáculos de forma reactiva.

4.2. Piloto: Agente de Navegación (Bajo Nivel)

El Piloto constituye el núcleo de inteligencia reactiva del sistema. Implementado en RobotScript.cs, es un agente de ML-Agents que hereda de la clase Agent y sobrescribe los métodos estándar: `Initialize()`, `OnEpisodeBegin()`, `CollectObservations()` y `OnActionReceived()`. Su responsabilidad exclusiva es conducir el AGV desde su posición actual hasta la coordenada del TargetGhost de forma suave, eficiente y sin colisiones.

4.2.1. Control Cinemático del AGV

El control del movimiento se realiza mediante la asignación directa de las propiedades de velocidad del componente Rigidbody de Unity (`rb.linearVelocity` y `rb.angularVelocity`), en lugar del enfoque convencional basado en fuerzas (`AddForce/AddTorque`). Esta decisión de diseño garantiza un control determinista y predecible de la cinemática del vehículo, eliminando la deriva y las oscilaciones inherentes a los sistemas basados en fuerzas aplicadas sobre cuerpos con inercia. El modelo 3D del vehículo y la configuración de su Rigidbody —masa, gravedad y constraints— se recogen en la Figura 4.2.

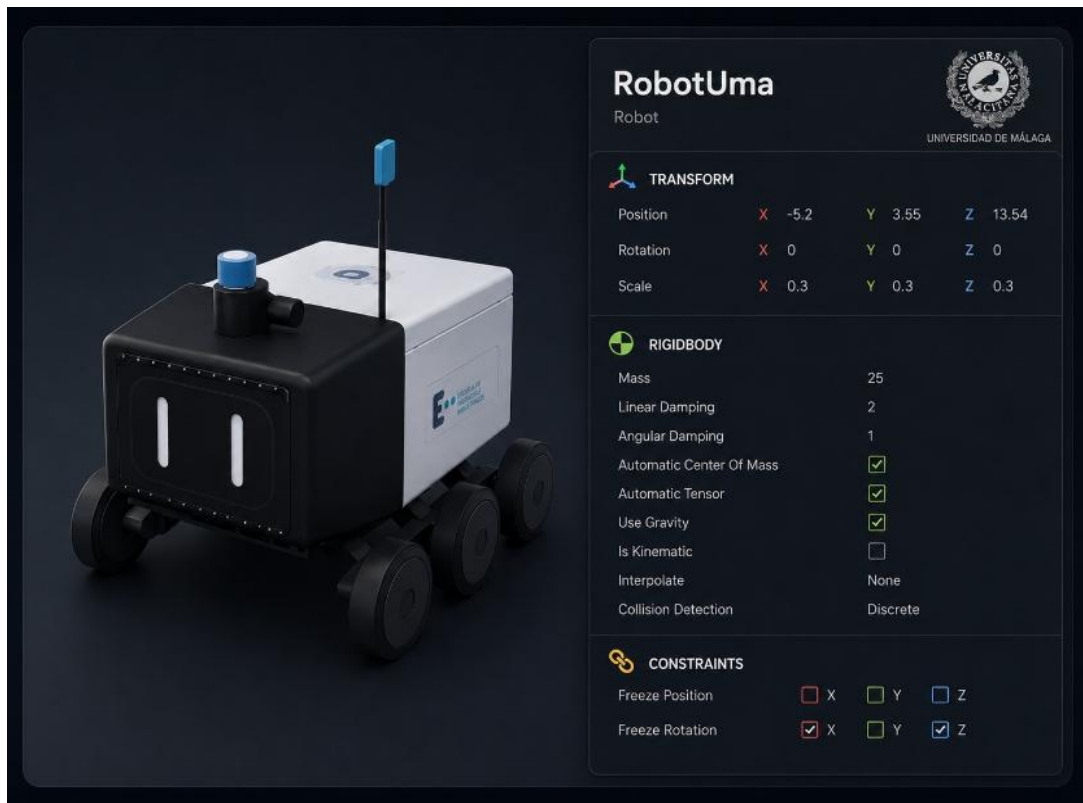


Figura 4.2: Modelo 3D y Configuración del Rigidbody en el Inspector: masa, gravedad, constraints

El AGV se configura con una masa de 25 kg, una velocidad lineal máxima de 1.5 m/s y una velocidad angular máxima de 200°/s. La gravedad permanece activa para que el robot se asiente sobre el suelo de forma natural, pero se congelan las rotaciones en los ejes X y Z del Rigidbody de Unity (un componente físico integrado que somete al objeto a las leyes dinámicas simulando masa, inercia y gravedad, impidiendo desplazamientos y rotaciones no controlados por código) para impedir que las colisiones vuelquen el chasis. Únicamente se permite la rotación libre en el eje Y (yaw), emulando el comportamiento de giro sobre su propio eje característico de este tipo de vehículos robotizados.

Una restricción fundamental es la eliminación de la marcha atrás. La acción continua de aceleración lineal, que la red neuronal emite en el rango estándar $[-1, 1]$, se remapea internamente al intervalo $[0, 1]$ antes de aplicarse como velocidad. De este modo, aunque la red solicite un valor negativo, el robot permanece inmóvil (0) o avanza (1), obligándolo a resolver cualquier situación mediante rotación y avance, sin retroceder. Esta restricción replica el comportamiento de un AGV industrial real y simplifica el espacio de soluciones que la política debe explorar, permitiendo un entrenamiento y su convergencia de valores mucho más acelerado.

4.2.2. Espacio de Observaciones

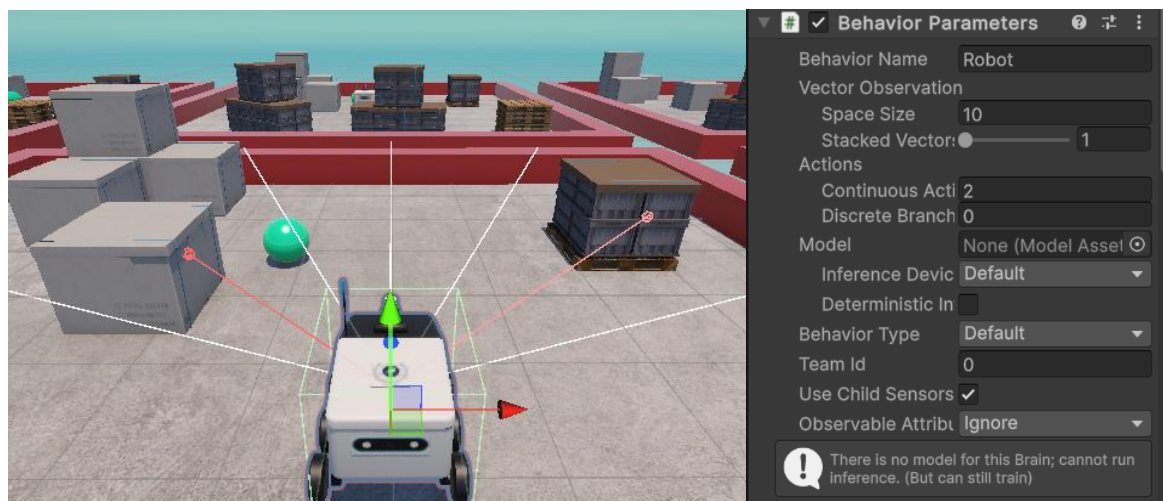


Figura 4.3: Espacio de observaciones del Piloto: vector local y Ray Perception Sensor.

El espacio de observaciones del Piloto se compone de dos fuentes de información complementarias (ver figura 4.3): un vector numérico de 10 componentes y un sensor de rayos (RayPerceptionSensor3D) que simula un LiDAR. El vector de observación se construye íntegramente desde la perspectiva del propio robot (coordenadas locales o egocéntricas), utilizando las herramientas de transformación de Unity (*InverseTransformPoint* e *InverseTransformDirection*).

Esta decisión de diseño es fundamental: al hacer que el robot evalúe su entorno basándose en 'lo que tiene a su alrededor' en lugar de usar coordenadas globales del mapa, la red neuronal no memoriza posiciones absolutas. Esto permite que el comportamiento aprendido sea invariante y se pueda generalizar fácilmente a entornos o fábricas no vistos durante el entrenamiento.

En la práctica, este vector se compone de 10 variables continuas (*floats*):

- **Posición relativa del objetivo:** 3 valores (X, Y, Z).
- **Dirección normalizada hacia el objetivo:** 3 valores.
- **Velocidad lineal local del robot:** 3 valores.
- **Velocidad angular actual:** 1 valor (rotación sobre su eje vertical).

El sensor `RayPerceptionSensor3D` complementa las observaciones vectoriales con información espacial del entorno inmediato. Funciona emitiendo un conjunto configurable de rayos desde el chasis del robot, detectando las etiquetas `Wall`, `Obstacle` y `Goal` en sus impactos. Cada rayo reporta la distancia normalizada al primer objeto detectado y la etiqueta correspondiente. El `Sphere Cast Radius` se ajusta a 0.1 para maximizar la precisión, y los offsets verticales de inicio y fin se calibran a 0.5 para evitar falsos positivos por contacto con el suelo. Es importante destacar que este sensor lleva configurado desde la primera fase de entrenamiento (E1), incluso cuando el entorno carece de obstáculos, ya que su presencia modifica la arquitectura de entrada de la red neuronal y añadirlo posteriormente invalidaría todos los pesos aprendidos.

4.2.3. Espacio de Acciones

El Piloto opera con un espacio de acción continuo bidimensional, configurado en los parámetros de `ML-Agents` como 2 acciones continuas. La primera acción controla la aceleración lineal (avance) y la segunda controla la velocidad angular (giro). Ambas acciones se emiten por la red neuronal en el rango $[-1, 1]$ y se escalan internamente por las velocidades máximas correspondientes (1.5 m/s y 200°/s). Como se ha descrito, la acción lineal se remapea adicionalmente a $[0, 1]$ para eliminar la marcha atrás.

La elección de un espacio de acción continuo frente a uno discreto es fundamental para la suavidad del movimiento. Un espacio discreto limitaría al robot a un conjunto finito de velocidades y ángulos predefinidos, produciendo trayectorias escalonadas y poco naturales. El espacio continuo permite al agente modular con precisión infinitesimal tanto la aceleración como el giro, resultando en trayectorias fluidas y curvas suaves que se aproximan al comportamiento de un conductor humano experto.

4.2.4. Sistema de Recompensas

El diseño final de la función de recompensa para el Piloto combina señales densas (para guiar el aprendizaje en pasillos largos) con penalizaciones reactivas ante colisiones y tiempo de ejecución. La función de recompensa final se compone de los siguientes términos:

- **Recompensa de Éxito:** +1.0 al alcanzar el objetivo (`TargetGhost`) con una orientación frontal.

- **Penalización por Colisión No Letal:** un castigo discreto de -0.5 en el instante del impacto inicial, sumado a un castigo continuo de -0.01 por cada frame en el que se mantenga el contacto físico con cualquier obstáculo, forzando la maniobra de desatasco.
- **Penalización por Desgaste Temporal:** un coste constante de -0.0005 por cada paso (step) de simulación para incentivar la rapidez del trayecto.
- **Reward Shaping por Delta de Distancia:** una recompensa de $r_t = d_{t-1} - d_t$, donde d_t es la distancia normalizada al objetivo en el instante t . Esto proporciona una señal de gradiente continua y orientada en pasillos largos, acelerando la convergencia. Cada paso en la dirección correcta recibe un pequeño incentivo, si se aleja se le penaliza.

Una vez conseguido un agente capaz de generalizar, adaptarse y navegar en cualquier entorno se da por concluido el entrenamiento del Piloto, por lo que se procedió al diseño del entrenamiento del nivel superior.

4.3. Gestor: Agente de Coordinación (Alto Nivel)

La toma de decisiones de alto nivel y la optimización del flujo logístico del almacén recaen sobre el **Agente Gestor**, implementado en la clase `GestorAgent.cs`. Este componente constituye el núcleo decisional de la arquitectura jerárquica, consistente en un modelo de Aprendizaje por Refuerzo Profundo entrenado mediante el algoritmo PPO (*Proximal Policy Optimization*) bajo el framework de Unity ML-Agents. El Gestor opera de forma totalmente desacoplada de la física y el movimiento cinemático de los AGVs; su función es recibir periódicamente el estado lógico consolidado de la planta desde la capa de simulación y seleccionar, para cada robot, la acción de asignación o desvío óptima en cada paso de decisión. El espacio de observaciones resultante del Gestor queda representado en la Figura 4.4.

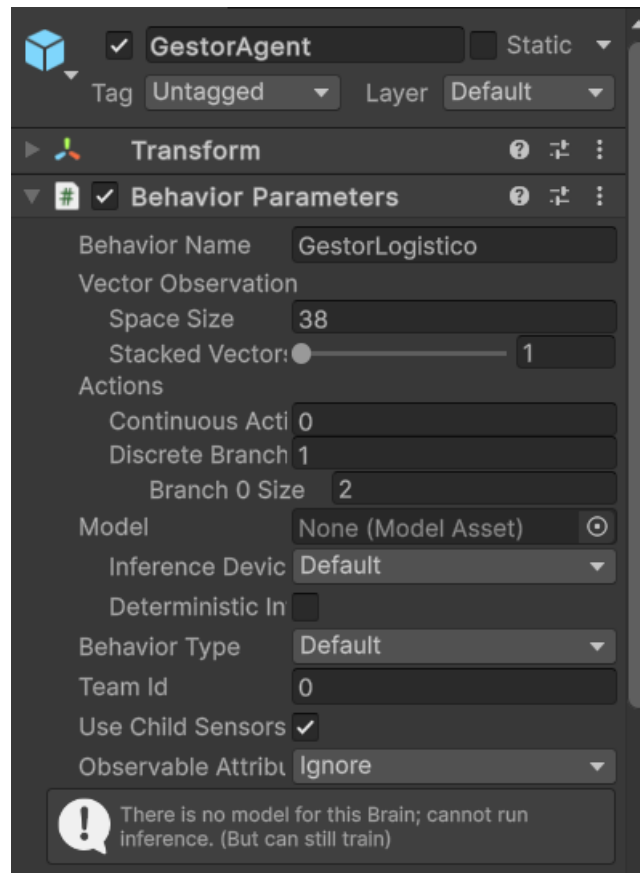


Figura 4.4: Espacio de observaciones del Gesto

r

4.3.1. Espacio de Observaciones

El vector de observaciones recopila exactamente **38 valores numéricos (de tipo float)** para dotar al agente de una visión global del estado lógico y espacial del almacén y sus robots:

- **Estado de las colas (2 floats):** longitud de las colas de espera en la Entrada 1 y la Entrada 2, normalizadas dividiendo por la capacidad máxima esperada ($C_{max} = 16$ paquetes).
- **Estado de la flota de robots (24 floats):** para cada uno de los dos robots se recopila:
 - **Estado lógico (5 floats):** codificación de activación única (*one-hot*) de la fase operativa en la que se encuentra el robot (Libre, Navegando A Recogida, Recogiendo, Navegando A Entrega, Entregando).

- **Entrada asignada (3 floats):** codificación *one-hot* del punto de recogida asignado (Ninguno, Entrada 1, Entrada 2).
- **Salida asignada (4 floats):** codificación *one-hot* de la zona de entrega asignada (Ninguna, Salida A, Salida B, Salida C).
- **Tiempos de espera críticos (2 floats):** tiempo de espera del paquete más antiguo (en el frente de la cola) en la Entrada 1 y Entrada 2, normalizado por un tiempo de espera máximo (segundos).
- **Previsión de destinos en colas (6 floats):** codificación *one-hot* de la zona de salida de destino (Salida A, B o C) del paquete que se encuentra en la cabeza de la cola 1 y la cola 2. Si la cola está vacía, se envía un subvector de ceros. Esto permite al Gestor prever el coste de oportunidad geográfico de cada asignación.
- **Posición del robot solicitante (2 floats):** coordenadas relativas (x,z) normalizadas (divididas por el tamaño del mapa de 80 x 80m) del AGV específico que ha pausado la simulación para requerir una decisión de enrutamiento.
- **Identificador del robot solicitante (2 floats):** codificación *one-hot* del ID del robot que requiere la acción [1, 0] para el vehículo AGV 0 y [0, 1] para el vehículo AGV 1, asegurando que la red asocie las coordenadas espaciales al robot correcto.

Nota sobre la Codificación de Variables Categóricas (Activación Única / One-Hot)

Para variables categóricas como los objetivos de recogida y entrega de cada robot, se implementa una **codificación de activación única** (también conocida en la literatura como codificación *one-hot* o *one-hot encoding*). En lugar de alimentar a la red neuronal con valores enteros secuenciales (por ejemplo, asignar el número 1 a la Salida A, el 2 a la Salida B y el 3 a la Salida C), se transforma cada categoría en un vector binario.

Si se utilizaran enteros secuenciales (1, 2, 3), la red neuronal interpretaría estas entradas en escala ordinal, asumiendo matemáticamente que la Salida C (3) es tres veces más importante o lejana que la Salida A (1), o que la Salida B (2) está a medio camino entre ambas en términos numéricos absolutos. Esto introduciría un sesgo artificial de prioridad o distancia que no se corresponde con la realidad física y logística del almacén. Para evitar

este sesgo, se crea un vector binario donde solo una posición toma el valor de 1 (activada) y el resto toma el valor de 0:

- Ningún objetivo activo: [1, 0, 0, 0]
- Salida A: [0, 1, 0, 0]
- Salida B: [0, 0, 1, 0]
- Salida C: [0, 0, 0, 1]

De este modo, todos los objetivos de entrega son ortogonales entre sí y equidistantes en el espacio vectorial del agente, lo cual garantiza que la red neuronal aprenda su codificación espacial de manera equitativa e independiente.

4.3.2. Espacio de Acciones Discretas

El espacio de acciones del Agente Gestor se diseña como un **espacio discreto unidimensional** de tamaño 2 por cada robot en el paso de decisión:

- **Acción 0 (Entrada 1):** asignar o desviar al AGV hacia el punto de Entrada 1.
- **Acción 1 (Entrada 2):** asignar o desviar al AGV hacia el punto de Entrada 2.

A diferencia del Piloto, que opera sobre un espacio continuo para modular aceleraciones físicas, el Gestor maneja decisiones lógicas de asignación. Para optimizar el entrenamiento y asegurar la viabilidad de las decisiones, se han integrado dos mecanismos en el software de control:

1. **Acción No-Op Implícita:** si el agente decide enviar un AGV a una entrada a la que ya estaba previamente asignado, el sistema físico detecta que no hay cambio de destino y el robot continúa su marcha actual sin reiniciar su trayectoria, actuando como una inacción (*no-op*).
2. **Action Masking (Enmascaramiento de Acciones):** mediante la sobreescritura del método `WriteDiscreteActionMask()`, se deshabilitan dinámicamente las acciones que dirigen a un robot libre hacia una cola vacía. Esto previene que el agente desperdicie pasos de entrenamiento seleccionando decisiones inválidas, acelerando notablemente la convergencia del aprendizaje.

4.3.3. Función de Recompensa

Función de Recompensa: se otorga un premio positivo (+1.0) al completar un ciclo logístico con éxito (paquete entregado en la salida). A su vez, se aplica un coste continuo de tiempo (-0.005 por paso) penalizado por cada paquete que espera en la cola, forzando a la red a evacuar las colas con rapidez y balancear dinámicamente el trabajo entre los dos puntos de entrada.

El diseño de la función de recompensa del Agente Gestor combina recompensas de éxito en hitos logísticos con penalizaciones continuas de tiempo y penalizaciones discretas por ineficiencias o errores en la asignación. Su estructura se define mediante los siguientes términos:

- **Recompensa Principal por Entrega (+1.0):** se otorga un premio positivo de +1.0 al completar con éxito un ciclo logístico completo (entrega de un paquete en su zona de salida correspondiente).
- **Recompensa Parcial por Recogida (+0.1):** se premia al agente con +0.1 en el instante en que un robot realiza la recogida física del paquete en el punto de entrada, reforzando la fase intermedia del ciclo.
- **Penalización por Acción Inválida (-0.5):** si el agente toma la decisión de enviar un AGV a un punto de entrada cuya cola de paquetes está vacía, se le aplica un castigo discreto de -0.5 (por ejemplo, si por seguridad el *Action Masking* tuviese que ser puenteado al estar ambas colas vacías).
- **Penalización por Tiempo de Espera Acumulado en Cola (Señal Densa):** se calcula en cada paso un coste continuo equivalente a $-0.0001 \times W \times \Delta t$, donde W es la suma de los tiempos de espera (en segundos) de todos los paquetes actualmente retenidos en ambas colas de entrada, y Δt es la duración del paso de simulación, también en segundos. El factor Δt garantiza que la penalización total acumulada a lo largo del tiempo no dependa de la frecuencia de decisión del agente: cuanto más fino es el paso, más veces se aplica pero menor es cada aplicación, de modo que el coste integrado en un mismo intervalo real es siempre el mismo. Así, la señal fuerza al Gestor a priorizar las colas más congestionadas y a balancear la carga de trabajo.
- **Penalización por Saturación / Desbordamiento (-2.0):** para evitar la acumulación infinita de inventario, si la suma de paquetes en cola es mayor o igual al límite de

desbordamiento (16 paquetes), el episodio se da por finalizado prematuramente con una penalización severa de -2.0.

- **Penalización Final por Paquetes Pendientes (-0.1 por paquete):** al finalizar de forma regular un episodio de simulación (duración de 8 minutos), el agente recibe una penalización final de -0.1 por cada paquete que haya quedado abandonado en las colas sin ser atendido.

4.4. Simulador de Almacén (Capa Lógica)

El entorno dinámico de la planta y el comportamiento operacional del flujo logístico se modelan mediante la **Capa de Simulación del Almacén (Capa Lógica)**. Este componente no alberga algoritmos de aprendizaje por refuerzo, sino que proporciona la infraestructura de software tradicional en C# necesaria para gestionar las colas de paquetes, registrar los tiempos de espera y coordinar las variables de estado. Esta capa actúa como el nexo que alimenta al Agente Gestor con observaciones lógicas y traduce sus decisiones en coordenadas físicas concretas. Estructuralmente, se divide en dos componentes principales:

- **Generador de Pedidos (GeneradorPedidos.cs):** encargado de la modelización matemática y estocástica de la demanda mediante procesos de Poisson.
- **Lógica del Almacén (GestorAlmacen.cs):** responsable de mantener el estado operativo de las colas, las fases de servicio de los robots y el ciclo de vida de los paquetes.

Esta clara separación conceptual permite independizar por completo la simulación operativa pura de las capacidades decisionales del agente de Inteligencia Artificial. Ambas clases, `GeneradorPedidos.cs` y `GestorAlmacen.cs`, aparecen con sus parámetros expuestos en el editor de Unity en la Figura 4.5.

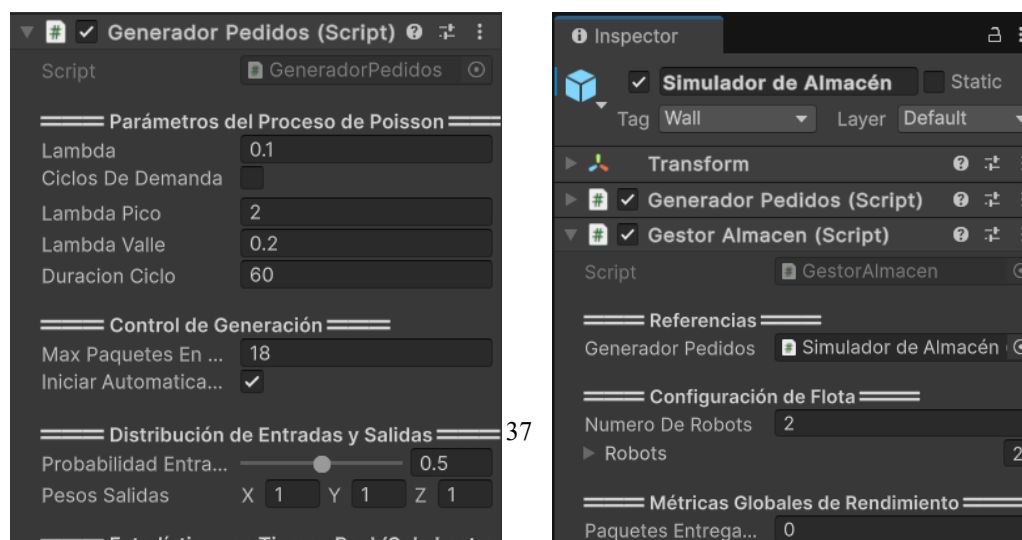


Figura 4.5: *GeneradorPedidos.cs* y *GestorAlmacen.cs* junto con sus parámetros en el editor de Unity

4.4.1. Generador de Pedidos (GeneradorPedidos.cs)

La demanda de paquetes se modela estocásticamente en la clase *GeneradorPedidos.cs* mediante un proceso de Poisson homogéneo. Los intervalos de tiempo entre arribos sucesivos se calculan aplicando el método de la transformada inversa a una distribución exponencial: $T = -\ln(U)/\lambda$, donde U es una variable aleatoria uniforme en $(0,1)$ y λ representa la tasa media de llegadas (paquetes por segundo). La Figura 4.6 ilustra el comportamiento de un proceso de Poisson para distintos valores de λ .

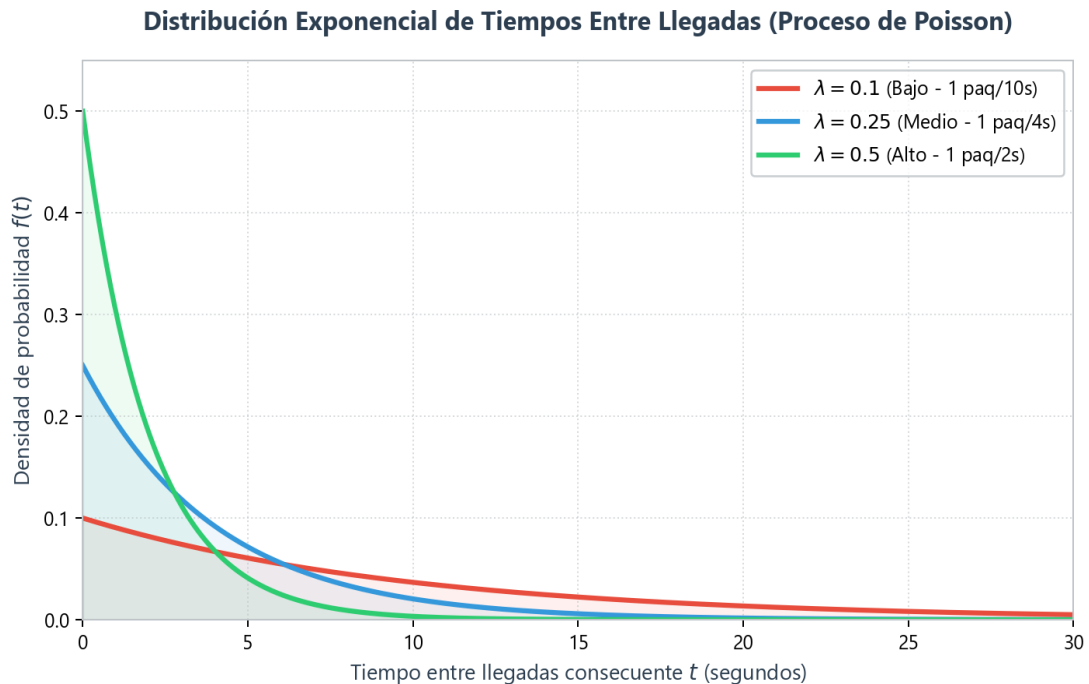


Figura 4.6: *Proceso de Poisson a distintos valores de lambda.*

El parámetro λ es configurable en tiempo real desde el Inspector, lo que permite evaluar el rendimiento ante diferentes niveles de carga (baja, media o alta). La asignación de la zona de salida de entrega (Salida A, B o C) se determina mediante muestreo ponderado para simular flujos de destino asimétricos, y un mecanismo de seguridad suspende la generación si se supera el límite físico de almacenamiento en las colas.

4.4.2. Lógica del Almacén (GestorAlmacen.cs)

La Lógica del Almacén es el componente del simulador encargado de llevar el registro detallado en tiempo real de los tiempos de espera, los paquetes y el estado operacional de la flota de robots. Implementada en la clase `GestorAlmacen.cs` (no es un agente, el nombre se mantiene en el código por razones de compatibilidad del editor), esta lógica gobierna el estado operacional del sistema de eventos discretos (DES) sin recurrir a físicas de Unity: administra las colas de paquetes en cada entrada, monitoriza las fases de servicio de los robots (Libre, NavegandoARecogida, Recogiendo, NavegandoAEntrega, Entregando) representados mediante instancias de `RobotInfo.cs`, y gestiona las transiciones lógicas del almacén.

Este componente de la Lógica del Almacén opera mediante un flujo orientado a eventos: monitorea de forma reactiva los eventos del Generador de Pedidos para registrar la llegada de paquetes a las colas y, a su vez, expone métodos públicos (como `AsignarRobotAEntrada()`, `ReportarLlegadaARecogida()` y `ReportarLlegadaAEntrega()`) que permiten al puente físico-lógico notificar las transiciones del mundo físico. Cada transición de estado actualiza el ciclo de vida del paquete y registra timestamps de alta precisión que alimentan el cálculo de métricas de rendimiento logístico.

4.4.3. Modelo de Datos (Paquete.cs)

La clase `Paquete.cs` define el objeto que almacena la información de cada envío: ID único, punto de entrada, destino, estado dentro de su ciclo de vida y datos temporales del ciclo. El ciclo de vida sigue las etapas: `EnCola`, `Asignado`, `EnTransitoRecogida`, `Recogido`, `EnTransitoEntrega` y `Entregado`. El registro preciso de los tiempos en cada estado permite calcular métricas como el tiempo en cola, tiempo de tránsito y tiempo total en el sistema, sirviendo como fundamento de optimización logística.

4.4.4. Cola de Decisiones y Reglas de Reasignación Dinámica

Para asegurar la viabilidad técnica y evitar atascos lógicos en Unity, se han diseñado dos mecanismos clave:

1. Cola de Decisiones Concurrentes: puesto que múltiples robots pueden requerir una decisión logística en el mismo frame de física, ML-Agents podría sufrir de sobreescritura de variables de estado. Para solventar este problema, GestorAgent.cs incorpora un buffer interno del tipo `Queue<int>`. Las peticiones de decisión de los robots se encolan de forma secuencial y se procesan una a una invocando a `RequestDecision()` de manera serializada.

2. Acotación de Reasignaciones (Desvíos en Caliente): la reasignación dinámica está permitida únicamente cuando el robot viaja vacío (estado `NavegandoAREcogida`). Para evitar oscilaciones infinitas de rumbo ("spam de decisiones"), se limita la petición de decisión a dos disparadores:

1 - Un robot queda libre.

2 – Entra un nuevo paquete en el frente de la cola. (Los nuevos en el buffer no activan esta decisión)

Si la cola ya tenía paquetes, el sistema asume que la asignación en curso ya ha sido optimizada. Si el agente decide reasignar un robot, este cambia de rumbo y el paquete queda liberado y por tanto vuelve a postularse como una opción de recogida.

4.5. Integración: Puente Físico-Lógico

Para que la toma de decisiones lógica de alto nivel (Gestor) se traduzca de forma efectiva en acciones físicas continuas en la fábrica (Piloto), se ha diseñado un puente de comunicación basado en eventos y una capa de coordinación lógica. Esta integración involucra tres componentes de software coordinados en Unity:

- **Lógica del Almacén (GestorAlmacen.cs):** componente de la capa de simulación que mantiene el estado de las colas, el ciclo de vida de los paquetes y los estados lógicos de la flota.

- **Agente Gestor de RL (GestorAgent.cs):** toma las decisiones de asignación/reasignación discreta de alto nivel basándose en observaciones lógicas del almacén.
- **Agente Piloto de RL (RobotScript.cs):** ejecuta el control continuo de motores físicos (velocidad lineal y angular) para conducir el AGV hacia el objetivo asignado.

4.5.1. Ciclo de Comunicación Orientado a Eventos

El flujo de control entre los distintos niveles de la jerarquía no se ejecuta mediante consultas continuas (*polling*), lo que saturaría el rendimiento del procesador, sino mediante un paradigma orientado a eventos con serialización de solicitudes:

- **Disparador de Evento:** cuando un AGV queda libre o un paquete nuevo entra en una cola, el script GestorAlmacen.cs dispara el evento OnRobotSolicitaDecision (robotId).
- **Encolamiento y Serialización:** la función de retorno denominada como: OnDecisionSolicitada en GestorAgent.cs captura el evento e inserta el ID del robot solicitante en un buffer interno de tipo Queue<int> (colaDecisionesPendientes). Esto evita problemas de concurrencia y sobreescritura de variables en ML-Agents si varios robots solicitan decisiones en el mismo frame temporal.
- **Consulta Neuronal:** si la cola tiene un solo elemento, se asigna robotIdPendiente e inmediatamente se invoca RequestDecision(), forzando a ML-Agents a ejecutar el ciclo de inferencia: CollectObservations() lee el vector de 38 observaciones lógicas y la red neuronal de nuestro **Agente de alto nivel** procesa el estado, emitiendo la acción en OnActionReceived().
- **Traducción Física:** la acción discreta elegida por el **Gestor** (Entrada 1 o Entrada 2) se traduce en coordenadas tridimensionales de Unity. El sistema desplaza el objeto fantasma (TargetGhost) del robot correspondiente a la posición de la entrada elegida.
- **Activación del Piloto:** el Gestor invoca el método IniciarMision() en el script RobotScript.cs del robot asignado, reiniciando su temporizador de vigilancia

y forzando a los motores físicos a navegar hacia las nuevas coordenadas del TargetGhost.

- **Cierre de Decisión:** finalmente, tras haber asignado un robot, se ejecuta el siguiente método `FinalizarYProcesarSiguienteDecision()`, que desencola la petición resuelta del buffer y si quedan más decisiones pendientes en la cola, asigna la siguiente e invoca de nuevo a `RequestDecision()`.

4.5.2. Flujo de Ciclo de Vida de un Paquete

El ciclo completo de procesamiento de un pedido en la arquitectura jerárquica sigue una secuencia de transiciones de estado lógico y físico:

1. **Generación:** el `GeneradorPedidos` crea un paquete y lo sitúa en una de las entradas lógicas del almacén.
2. **Asignación de Entrada:** el Gestor asigna un AGV a la entrada del paquete moviendo su TargetGhost. El AGV navega y realiza la recogida.
3. **Transición de Carga:** al llegar a la entrada, la lógica física acopla visualmente la caja sobre el chasis del AGV. El Gestor actualiza el TargetGhost a las coordenadas de la salida asignada a ese paquete (Salida A, B o C).
4. **Entrega y Liberación:** el AGV navega hacia la salida. Al llegar, se simula la entrega destruyendo el paquete visual, sumando una entrega exitosa y devolviendo al AGV al estado de Libre.

4.5.3. Controlador de Línea Base (FIFO Baseline)

Para validar cuantitativamente la eficiencia del Gestor inteligente, se ha implementado en paralelo un script controlador simplificado llamado “ControladorBasicoPrueba.cs”.

Este script prescinde de la red neuronal del Gestor y utiliza la misma interfaz de eventos para aplicar una regla secuencial **FIFO**: asigna siempre el paquete más antiguo disponible en las colas al primer robot que quede libre. Este controlador **no optimiza** las distancias ni realiza balanceos, sirviendo exclusivamente como la línea base estándar de comparación en los experimentos del Capítulo 5.

4.6. Adaptación del Piloto para Modo Producción

Para permitir que el modelo pre-entrenado funcione dentro del sistema logístico completo, fue necesario introducir un interruptor de modo en RobotScript.cs. El booleano ModoEntrenamiento controla dos comportamientos críticos:

- 1- **Cuando está activo**, el agente opera en el ciclo estándar de ML-Agents con reinicios de episodio, teletransporte aleatorio y recolección de recompensas.
- 2- **Cuando está desactivado** (modo producción), se suprimen las llamadas a EndEpisode() que normalmente reiniciarían la posición del robot, permitiéndole operar de forma continua bajo las órdenes del Gestor.

La red neuronal entrenada (.onnx) se integra sin modificaciones: sigue recibiendo exactamente las mismas observaciones (vector local de 10 componentes + rayos LiDAR) y emitiendo las mismas dos acciones continuas. El modelo no distingue si está en entrenamiento o en producción; simplemente reacciona a las observaciones que percibe. Esta transparencia de integración es una ventaja fundamental de la arquitectura jerárquica: el Piloto es un módulo intercambiable que puede ser reentrenado o sustituido sin afectar al resto del sistema.

5. Experimentos y Resultados

Este capítulo presenta los resultados obtenidos en cada fase experimental del proyecto, organizados cronológicamente siguiendo el curriculum learning emergente descrito en la Sección 3.2. Para cada fase se describe el objetivo específico, la configuración del entorno, las métricas de entrenamiento registradas en TensorBoard y el análisis crítico de los resultados. Se presta especial atención a los fallos detectados y cómo motivaron el diseño de la fase siguiente, ya que esta progresión iterativa constituye una de las aportaciones metodológicas principales del trabajo.

5.1. Fase E1: Navegación Pura

La primera fase de entrenamiento tiene como objetivo validar la capacidad fundamental del agente de bajo nivel: navegar desde su posición actual hasta una coordenada de destino en un espacio libre de obstáculos. El entorno consiste en una arena de 8×8 metros completamente vacía, donde el TargetGhost se teletransporta a una posición aleatoria al inicio de cada episodio. La distribución espacial de este entorno E1, con el AGV y el objetivo (TargetGhost) sobre un plano libre de obstáculos, se muestra en la Figura 5.1.

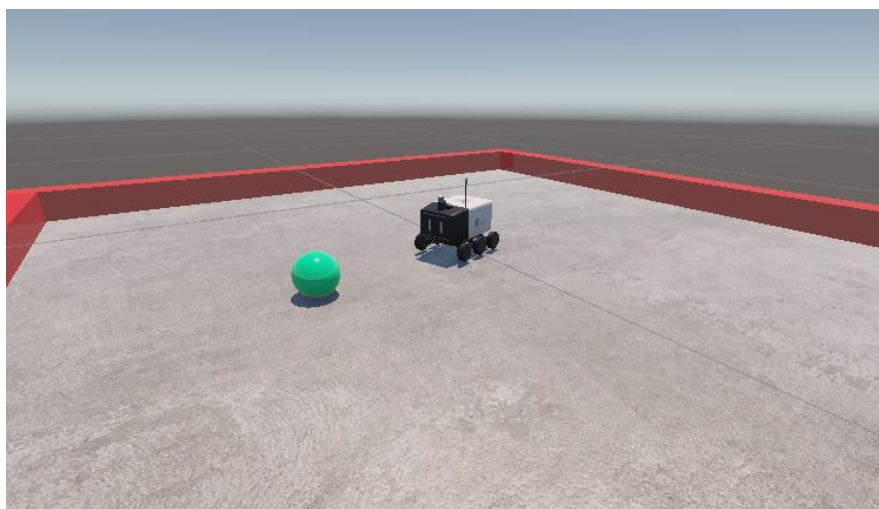


Figura 5.1: Distribución espacial del entorno de la Fase E1, con el AGV y el objetivo (TargetGhost) en un plano libre de obstáculos de 8×8 metros.

El entrenamiento se lanzó en Python ejecutando el siguiente comando: “`mlagents-learn config/E.1_SoloNavegacion.yaml --run-id=E.1_SoloNavegacion`”, configurado con un máximo de 10 millones de pasos. La función de recompensa utilizada es la V1 (sparse): +1.0 al alcanzar la meta con orientación frontal, -1.0 al colisionar con las paredes perimetrales, y -0.0005 por paso como coste temporal.

Resultado: el entrenamiento se detuvo manualmente de forma satisfactoria en torno a los 5.25 millones de pasos, habiendo demostrado una convergencia total. El modelo exportado fue Robot-5250000.onnx. Esta convergencia se observa con claridad en la Figura 5.2: la recompensa acumulada crece de forma sostenida hasta estabilizarse en su valor máximo, mientras que la longitud del episodio se reduce y se aplanas, señal de que el agente alcanza el objetivo cada vez en menos pasos.

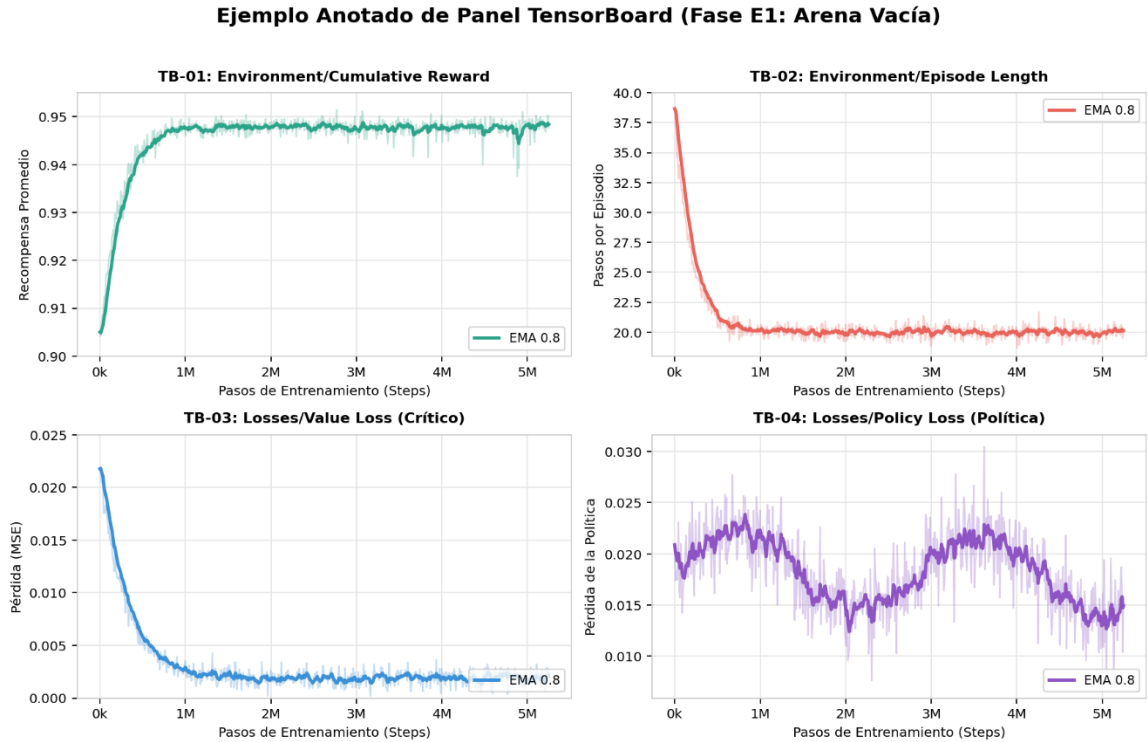


Figura 5.2: Resultados de TensorBoard correspondientes al entrenamiento de la Fase E1: Arena Vacía

(Recompensa Acumulada, Longitud del Episodio, Pérdida del Crítico y Pérdida de la Política).

5.1.1. Análisis de Métricas

Cumulative Reward: la curva muestra un ascenso logarítmico abrupto durante el primer millón de pasos, seguido de una estabilización en torno a 0.95. Teniendo en cuenta la penalización de desgaste de -0.0005 por step, este valor implica que el robot alcanza el objetivo en una media de 100 pasos por episodio, sin colisionar y trazando rutas prácticamente óptimas.

Episode Length: se observa un desplome inicial de la duración media y una consolidación estable en torno a 20 steps por episodio en la fase madura del modelo. Esto confirma la

eficiencia cinemática del agente: localiza su destino frontal con rapidez y no duda ni deambula.

Value Loss: decrece desde un pico inicial hasta magnitudes residuales (entre 0.0002 y 0.0019), indicando que el Critic predice la función de recompensa del entorno con alta precisión.

Policy Loss: muestra oscilaciones persistentes y sanas, propias del mecanismo de clipping de PPO ajustando probabilidades estables. No se observa divergencia.

La Fase E1 valida la arquitectura base del agente: el espacio de observaciones egocéntrico, el control cinemático directo y la restricción de marcha atrás producen un navegador eficiente y robusto en entornos abiertos.

5.2. Fase E2: Obstáculos Estáticos

La segunda fase introduce obstáculos fijos en la arena para forzar al agente a desarrollar capacidades de evasión. A diferencia de E1, tanto la posición del robot como la del TargetGhost se aleatorizan en cada episodio mediante spawn dinámico, utilizando Physics.OverlapSphere para garantizar que ninguno aparezca dentro de un obstáculo existente.

El entrenamiento de la Fase E2 se extendió hasta alcanzar los 15.0 millones de pasos acumulados (aportando 9.75 millones de pasos de entrenamiento específico en esta fase de obstáculos fijos). Al heredar los pesos del modelo maduro de la Fase E1, el agente partió con una base robusta de orientación y búsqueda de objetivos, lo que aceleró significativamente la velocidad de entrenamiento.

Por otro lado, la función de recompensa se mantiene idéntica a V1. La distribución espacial de la Arena E2 se representa en la Figura 5.3.



Figura 5.3: Distribución espacial de la Arena E2

A continuación, los resultados obtenidos en TensorBoard (Figura 5.4):

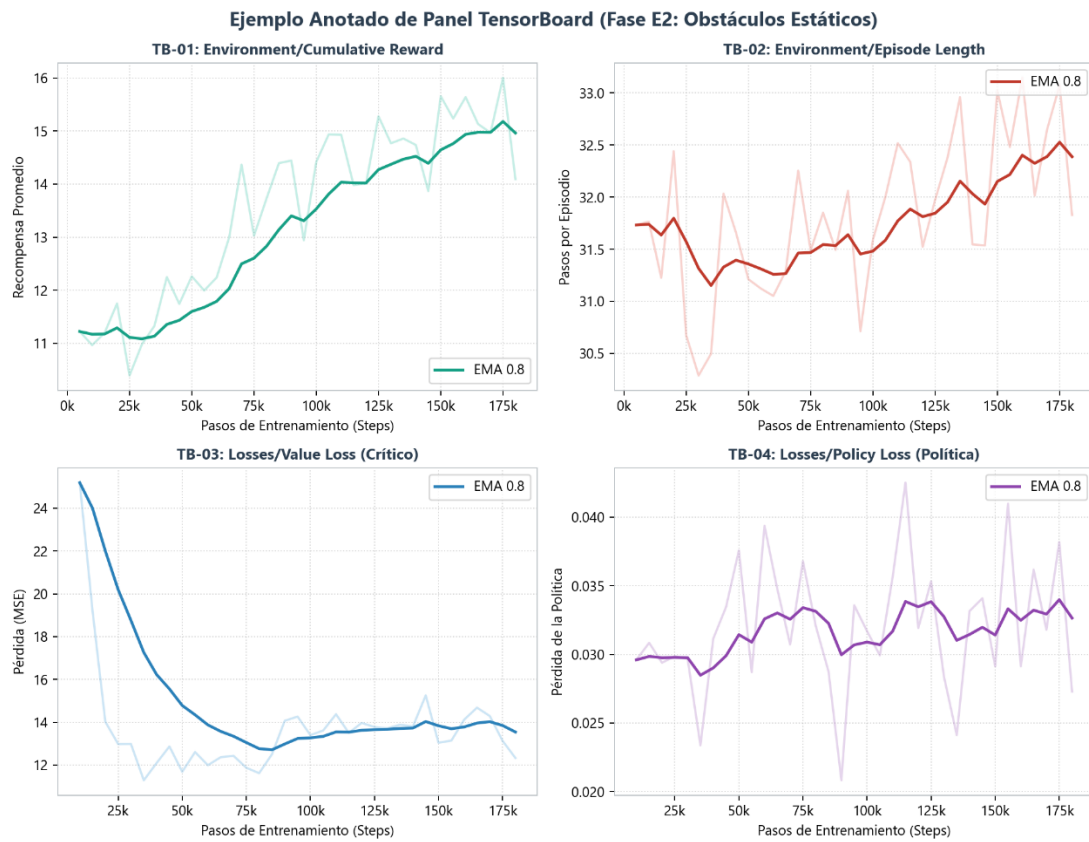


Figura 5.4: Resultados de TensorBoard correspondientes al entrenamiento de la Fase E2 con obstáculos estáticos (Recompensa Acumulada, Longitud del Episodio, Pérdida del Crítico y Pérdida de la Política).

Resultado: el modelo convergió consolidando una búsqueda de rutas reactiva y adaptativa. Al revisar el comportamiento en movimiento, se puede apreciar que el agente entrenado es capaz de rodear obstáculos estáticos de diversas formas y tamaños, manteniendo trayectorias suaves y eficientes.

Para preservar la reproducibilidad del proyecto, se adoptó una estrategia de versionado modular: cada fase se almacena como un Prefab independiente (Preset_Basico.prefab, Preset_Obstaculos.prefab) dentro de su propia escena de Unity, permitiendo restaurar cualquier configuración anterior a voluntad.

5.3. Fase E3: Entornos Procedurales

La tercera fase supone el salto cualitativo más importante del curriculum: el escenario físico completo muta en cada episodio. El script RandomObstaclesGenerator.cs redistribuye los obstáculos siguiendo una distribución gaussiana centrada (generada mediante la suma de dos variables uniformes) que concentra la densidad de obstáculos en el centro de la pista, forzando rutas complejas. Un algoritmo de validación de pasillo garantiza que siempre exista al menos una ruta transitable entre el robot y la meta.

El entrenamiento hereda los pesos de E2 y se ejecuta durante aproximadamente 38.7 millones de pasos, acumulando una experiencia masiva en configuraciones geométricas radicalmente diferentes en cada episodio. La distribución de la Arena E3, un entorno completamente aleatorio, se ilustra en la Figura 5.5.



Figura 5.5: Distribución espacial de la Arena E3, ilustrando un entorno aleatorio.

El entrenamiento hereda los pesos de E2 y se ejecuta durante aproximadamente 38.7 millones de pasos, acumulando una experiencia masiva en configuraciones geométricas radicalmente diferentes en cada episodio. Los resultados de este entrenamiento se muestran en la Figura 5.6.

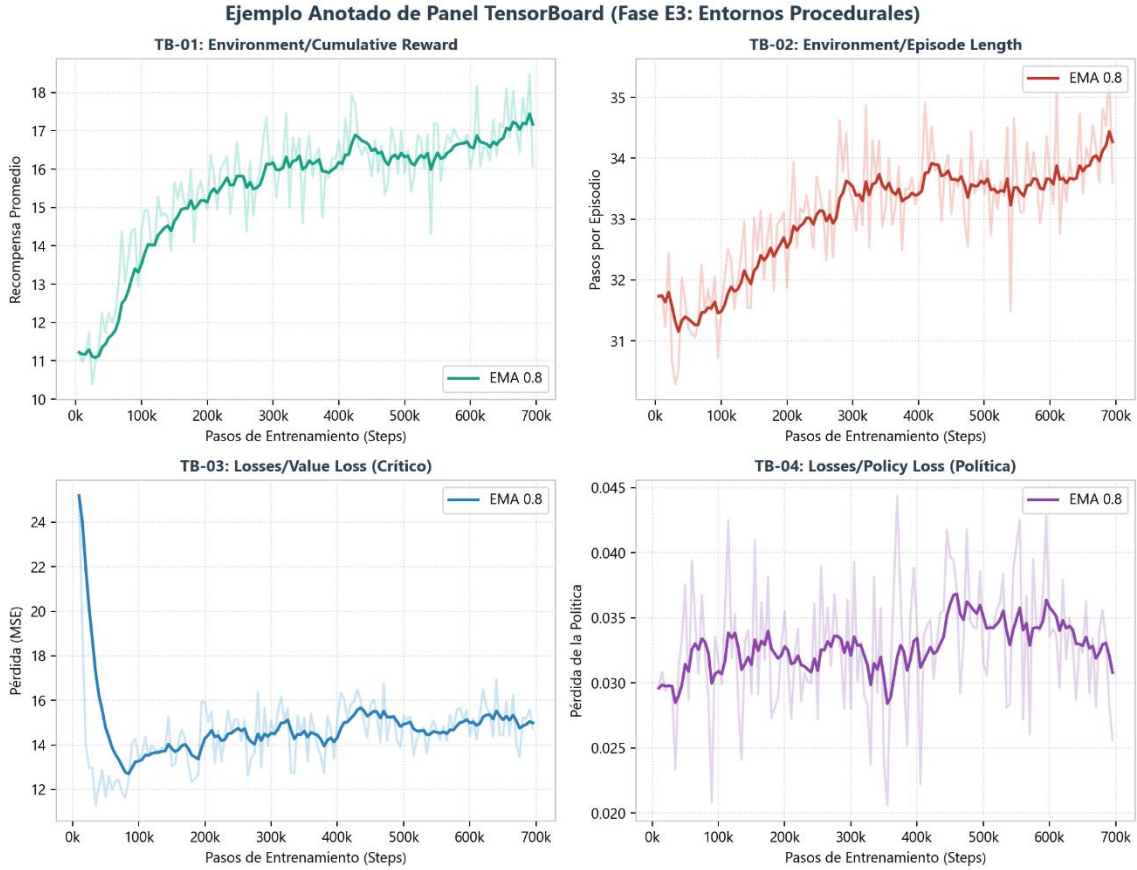


Figura 5.6: Resultados de TensorBoard correspondientes al entrenamiento de la Fase E3 en entornos procedurales dinámicos (Recompensa Acumulada, Longitud del Episodio, Pérdida del Crítico y Pérdida de la Política).

Resultado: el robot alcanza la meta esquivando los mapas generados proceduralmente un 99% de las veces, demostrando una generalización robusta a entornos no vistos previamente. Este resultado valida la eficacia del curriculum learning progresivo y la solidez del espacio de observaciones egocéntrico.

5.3.1. Detección del Mínimo Local (Visión de Túnel)

A pesar de la robustez general del modelo, se detectó un caso límite significativo: cuando la generación procedimental sitúa dos obstáculos dejando una rendija milimétrica a través de la cual el chasis físico del AGV no cabe, pero los sensores LiDAR sí detectan la meta al otro lado, el robot entra en un bucle infinito de intentos de penetración. Al percibir la distancia al objetivo como insignificante a través de la rendija, la política determina que insistir es matemáticamente óptimo frente al enorme coste temporal de retroceder y buscar un rodeo por un mapa del que no tiene visión global. La Figura 5.7 ilustra este problema de mínimo local o visión de túnel.

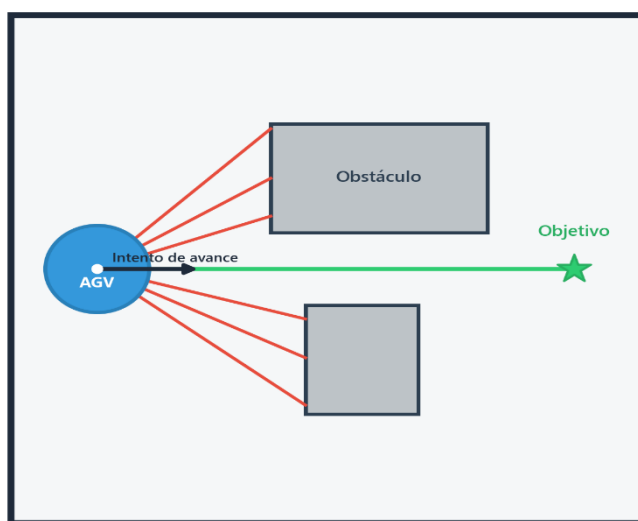


Figura 5.7: Ilustración del problema de mínimo local / visión de túnel, donde el AGV colisiona frontalmente contra un obstáculo en U al intentar avanzar en línea recta hacia la meta.

Finalmente, se determinó que el siguiente comportamiento es teóricamente correcto y predecible: un agente entrenado exclusivamente en navegación reactiva local, que decide fotograma a fotograma en función de percepciones parciales inmediatas, carece de la información necesaria para detectar callejones sin salida a escala macroscópica. Lejos de ser un fallo a depurar, este fenómeno constituye una justificación académica rigurosa de la necesidad de una arquitectura jerárquica: el Piloto de bajo nivel requiere una capa superior con visión global que le proporcione *waypoints* seguros, liberando a la red neuronal de la responsabilidad de planificación a largo plazo.

5.4. Fase E4: Resiliencia Post-Colisión

Las pruebas de integración del modelo V3 (38.7 millones de pasos) en la escena industrial revelaron un segundo problema estructural: el robot quedaba permanentemente atascado contra esquinas y pilares anchos, empujando en línea recta hacia la meta sin rectificar. La causa subyacente se identificó rápidamente: durante las fases E1–E3, la función de recompensa invocaba `EndEpisode()` instantáneamente al detectar una colisión, por lo que, en sus 39 millones de pasos de experiencia, la red neuronal jamás experimentó ni exploró el estado posterior a un choque. El robot sufría una ceguera espacial post-colisión.

Para resolver este problema, se diseñó la función de recompensa V3 con colisión no letal: un castigo de -0.5 en el impacto inicial (`OnCollisionEnter`) y un castigo continuo de -0.01 por cada frame de contacto sostenido (`OnCollisionStay`). El entrenamiento V4 se ejecutó durante toda una noche, heredando los pesos de V3, y alcanzó los 22.83 millones de pasos. El contraste entre la señal en bruto y la suavizada se aprecia en las Figuras 5.8 y 5.9:

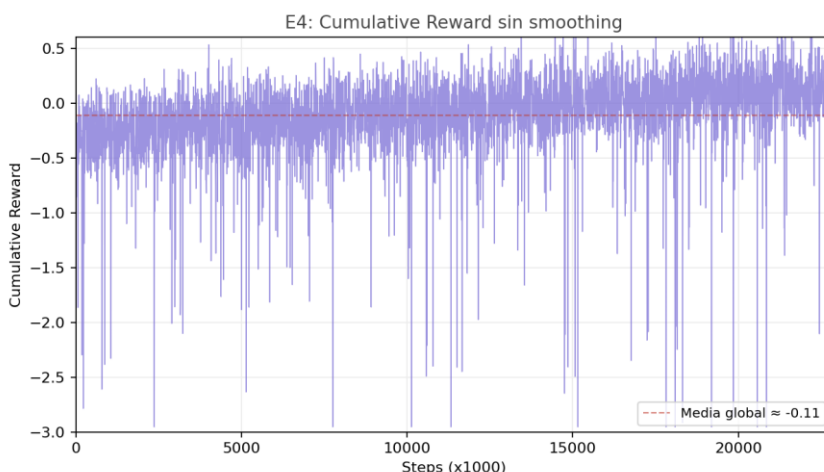


Figura 5.8: E4: Cumulative Reward sin suavizado — señal caótica con alta varianza.

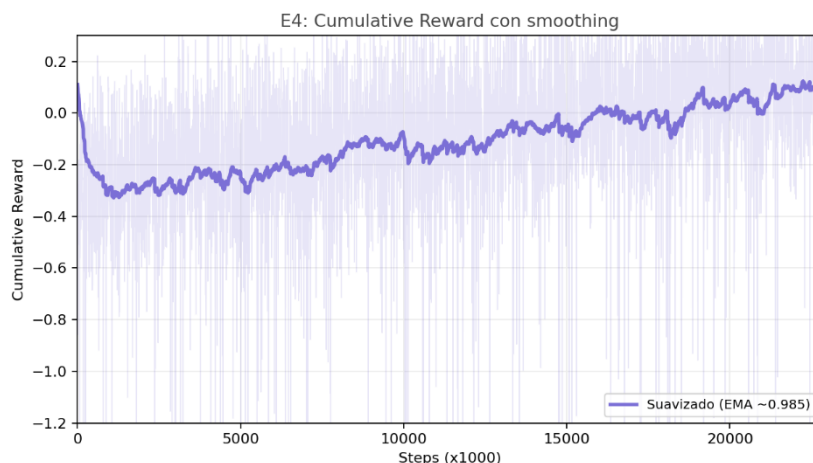


Figura 5.9: E4: Cumulative Reward con suavizado 0.999 — tendencia ascendente revelada.

5.5. Fase E5: Optimización de Trayectorias (Guiado Suave)

Tras el éxito en la evitación de obstáculos dinámicos en entornos procedurales y la incorporación de resiliencia post-colisión (Fase E4), se observó que el agente tendía a realizar un patrón de navegación con oscilaciones laterales constantes (zigzag) en pasillos largos. Para mitigar esta ineficiencia cinemática, la Fase E5 tuvo como objetivo optimizar la trayectoria del AGV. El entrenamiento se inició heredando los pesos de E4 (E4_RectificarColisiones) y aplicando una función de recompensa V5 que añade una penalización por giros bruscos y aceleración angular excesiva. El entrenamiento progresó hasta alcanzar los 80 millones de pasos acumulados. **Resultado:** el agente aprendió a corregir la deriva de forma proactiva, logrando trayectorias lineales fluidas en tramos rectos sin zigzag, lo que redujo el desgaste mecánico simulado del vehículo a costa de una velocidad lineal promedio ligeramente menor en curvas.

5.5.1. Evolución del Sistema de Recompensas (V1 a V5)

El diseño de la función de recompensa constituye uno de los aspectos más críticos e iterativos del proyecto. A lo largo de las seis fases experimentales, el sistema de recompensas evolucionó cinco veces (V1–V5), cada versión motivada por una limitación concreta detectada en el comportamiento del agente.

V1 — Recompensa Sparse Básica (E1): la primera versión implementa un esquema minimalista de tres señales: +1.0 al alcanzar el TargetGhost con orientación frontal (para evitar que adoptara soluciones en las que se desplace girando sobre su propio eje), -1.0 con fin de episodio inmediato al colisionar con cualquier obstáculo (Wall, Obstacle), y un coste temporal de -0.0005 por paso para penalizar la lentitud y la inactividad. Este diseño es suficiente para entornos simples donde la distancia al objetivo es corta.

V2 — Mantenimiento en E2 y E3: las fases de obstáculos fijos (E2) y procedurales (E3) mantienen el esquema V1 sin modificaciones. La complejidad creciente del entorno proporciona un aprendizaje con un incremento de dificultad natural que fuerza la generalización sin necesidad de alterar las señales de recompensa. En E3, el modelo alcanza un 99% de éxito en mapas aleatorios, validando la robustez del enfoque sparse en entornos de escala controlada.

V3 — Colisión No Letal (E4): el análisis del despliegue en la fábrica reveló que el robot carecía de maniobras de rectificación post-colisión, ya que durante el entrenamiento con V1/V2 el episodio terminaba instantáneamente al chocar. La versión V3 sustituye la muerte súbita por un castigo de -0.5 en el impacto inicial (OnCollisionEnter) y un castigo continuo de -0.01 por cada frame de contacto sostenido (OnCollisionStay). Este desangrado matemático fuerza al agente a explorar activamente maniobras de escape, interiorizando por primera vez giros drásticos y pivoteos para liberarse de geometrías complejas.

V4 — Normalización Dinámica Contextual (E5): al desplegar el modelo en la fábrica de 80×80 metros, las observaciones brutas saturaban las funciones de activación de la red neuronal (Out-of-Distribution Shift). La solución consistió en normalizar el vector de distancia al objetivo dividiéndolo por la distancia inicial registrada al comienzo de cada misión (IniciarMision()). La red recibe así valores acotados en [-1, 1] independientemente del tamaño del mapa, percibiendo proporciones en lugar de magnitudes absolutas.

V5 — Reward Shaping por Delta de Distancia (E5/E6): para guiar al agente a través de las distancias extensas del entorno final de la fábrica (80×80 m), se incorporó una recompensa densa basada en la ganancia de distancia en cada paso: $r_t = d_{t-1} - d_t$, donde d_t es la distancia normalizada al objetivo en el paso t . De esta forma, el agente recibe un micro-premio continuo por cada paso que reduce la distancia a la meta (y una penalización si se aleja), eliminando el problema de la recompensa dispersa (sparse reward) y proporcionando una señal de gradiente constante a lo largo de todo el trayecto.

5.6. Despliegue en Entorno a Gran Escala

Con el modelo V4 validado en las arenas de entrenamiento, se procedió a su despliegue en el escenario final: la fábrica industrial de 80×80 metros. Esta transición de escala (10× respecto a las arenas de 8×8 metros) reveló un fenómeno crítico documentado en la literatura, pero que no supe tener en cuenta desde el principio: el Generalization Gap.

5.6.1. Detección del Generalization Gap

Al desplegar el modelo en la fábrica, el agente mostró un rendimiento errático e incapaz de replicar sus habilidades evasivas y direccionales. La investigación de la telemetría y las

observaciones del agente determinó que el problema no radicaba en la topología de los obstáculos, sino en un fenómeno clásico de Deep Learning: el Out-of-Distribution (OOD) Shift.

Durante el entrenamiento en arenas de 8×8 metros, las coordenadas del vector de distancia hacia la meta oscilaban en un rango aproximado de $[-8, 8]$. La red neuronal ajustó sus pesos para procesar estas magnitudes. Al desplegarse en la fábrica, los vectores de distancia alcanzaron rangos de $[-80, 80]$. Estos valores, inyectados sin normalizar en las funciones de activación, saturaban los tensores hacia sus límites, corrompiendo la salida de la red y provocando acciones erráticas.

5.6.2. Solución: Normalización Dinámica y Reward Shaping

Se aplicaron dos correcciones fundamentales. La primera fue la normalización dinámica contextual: el vector de distancia al objetivo se divide por la distancia inicial registrada al comienzo de cada misión (`IniciarMision()`), acotando las observaciones en $[-1, 1]$ independientemente del tamaño del mapa. La segunda fue la densificación de la recompensa mediante el cálculo de la diferencia incremental de la distancia al objetivo en instantes de tiempo consecutivos ($rt = dt-1 - dt$), de manera que el agente recibe una pequeña recompensa positiva cada vez que se acerca físicamente a la meta y una penalización equivalente cuando se aleja de ella, proporcionando una señal de gradiente continua en trayectos largos.

Adicionalmente, tras investigar el código se identificó un bug crítico en el modo producción: cuando `modoEntrenamiento` era false, la función `OnEpisodeBegin()` ejecutaba un return inmediato, lo que impedía que las variables `initialDistanceToGoal` y `previousDistanceToGoal` se recalculasen al cambiar de misión. La solución fue implementar el método público `IniciarMision()` con un detector automático en `CollectObservations()` que recalibra las variables cuando el `TargetGhost` salta más de 2 metros entre frames.

5.6.3. Resultado: Despliegue Exitoso sin Reentrenamiento

Tras aplicar únicamente el fix de normalización, el agente funcionó a la perfección en la fábrica real sin necesidad de reentrenar el modelo. Los pesos entrenados durante millones de steps en las arenas de 8×8 metros se transfirieron directamente al entorno de 80×80 metros, demostrando que el agente había aprendido comportamientos de navegación genuinamente generalizables. La causa raíz del fallo no era un problema de entrenamiento ni de arquitectura de red, sino un bug de integración en la capa de preprocesado de datos.

Este resultado constituye una de las aportaciones prácticas más relevantes del trabajo: la documentación detallada de cómo un fenómeno teórico conocido (OOD Shift) se manifiesta en un proyecto real, incluyendo el proceso de diagnóstico, las pistas que condujeron a la solución, y la importancia crítica del preprocesado de datos en arquitecturas de Reinforcement Learning.

5.7. Análisis de Comportamientos Emergentes

Durante la observación de las inferencias del modelo, se detectó un patrón de movimiento peculiar: en trayectos largos y despejados, el AGV no traza una línea matemáticamente recta, sino que oscila levemente describiendo esos o zigzags tenues (ver Figura 5.10). Este comportamiento, lejos de ser un error, se interpreta como un comportamiento emergente de escaneo espacial.

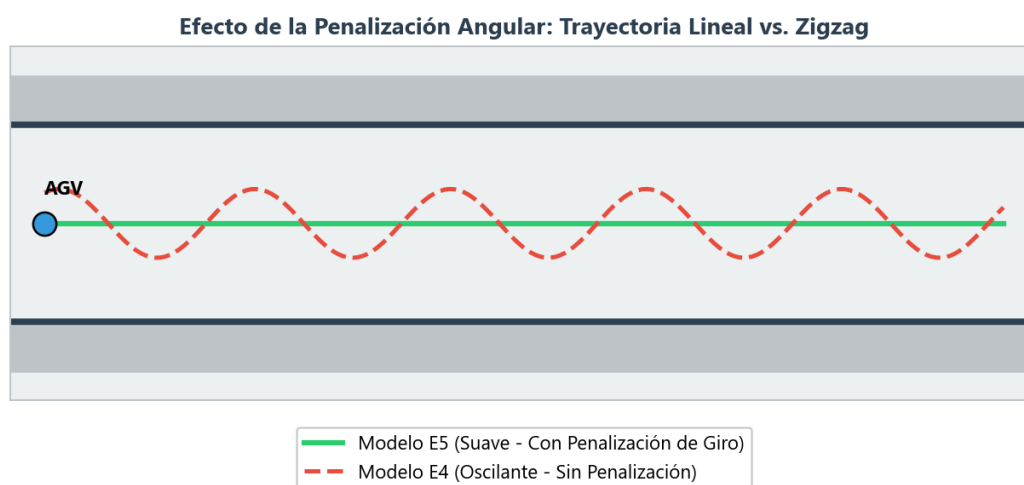


Figura 5.10: Comparativa de trayectorias en pasillo largo, (zigzag) del modelo E4 frente al guiado lineal suavizado del modelo E5 con penalización angular.

Los sensores RayPerceptionSensor3D del agente emiten rayos en ángulos fijos, lo que genera puntos ciegos entre rayo y rayo. La red neuronal aprendió de forma autónoma que oscilar el chasis direccionalmente produce un barrido continuo del entorno, similar a un radar rotativo. Esta estrategia le permite detectar obstáculos más estrechos que de otro modo pasarían desapercibidos entre sus haces de visión. Al no existir una penalización explícita por esfuerzo direccional (gasto energético en giro), el agente no encuentra incentivo matemático para trazar una línea perfectamente recta si el zigzag proporciona mayor seguridad perceptiva sin comprometer sustancialmente el tiempo de llegada.

Es importante señalar que este zigzag es un comportamiento determinista aprendido por la política, no ruido estocástico. Durante la inferencia, la red utiliza únicamente la media de su distribución de acciones por lo que cada acción es completamente determinista. El zigzag emerge porque la política óptima aprendida incluye esta oscilación como parte de su estrategia de navegación.

5.8. Análisis Comparativo de Modelos

La Tabla 5.1 resume el análisis comparativo de los modelos entrenados a lo largo de las fases:

Fase / Modelo	Pasos Totales	Objetivo de la Fase	Enfoque de Recompensa	Tasa de Éxito	Capacidad Clave	Limitación
E1 (Básico)	5.25M	Navegación básica en entorno vacío.	Densa por cercanía + Premio por llegar al objetivo.	~100% de éxito	Movimiento fluido y directo hacia el objetivo.	Incapacidad absoluta para esquivar obstáculos.
E2 (Estático)	15.0M	Evasión de obstáculos fijos (estáticos).	Penalización por colisión severa (colisión = fin).	~95% en estáticos	Evitación de obstáculos en línea de visión directa.	Atascamiento frecuente en esquinas o zonas estrechas.
E3 (Procedural)	38.7M (acumulados)	Evasión robusta en entornos procedurales.	Penalización por colisión severa.	99% de éxito	Generalización a mapas no vistos y obstáculos imprevistos.	Inestabilidad al rozar esquinas (atascamiento).

E4 (Resiliente)	61.5M (acumulados)	Navegación robusta con resiliencia ante colisiones.	Penalización reducida por contacto, premiando la recuperación.	~100% operativo	Autorecuperación y maniobras de desatascos autónomo.	Mayor coste computacional para la convergencia.
E5 (Guiado Suave)	80.0M (acumulados)	Optimización de trayectos y reducción de oscilaciones.	Penalización por giros bruscos y aceleración angular.	~100% operativo	Trayectorias lineales sin zigzag en pasillos largos.	Velocidad lineal promedio reducida ligeramente.

Tabla 5.1: Análisis Comparativo de Modelos

La progresión de capacidades a través de las fases experimentales demuestra la eficacia del curriculum learning. El modelo E1, con 5.25 millones de pasos, domina la navegación directa pero carece de capacidad evasiva. El modelo E2/E3, con 38.7 millones de pasos acumulados, generaliza a entornos procedurales con un 99% de éxito pero falla ante colisiones sostenidas. El modelo E4, con 22.8 millones de pasos adicionales, incorpora resiliencia post-colisión y maniobras de rectificación. Finalmente, el modelo E5 demuestra que la normalización dinámica permite transferir todas estas capacidades a un entorno $10\times$ mayor sin reentrenamiento. Cada modelo hereda y amplía las capacidades del anterior sin destruir las previamente adquiridas, validando que la arquitectura de la red (invariante en todas las fases) y el espacio de observaciones egocéntrico proporcionan una base lo suficientemente general para soportar especialización progresiva.

5.9. Fase E6: Entrenamiento del Gestor de Alto Nivel

La sexta y última fase experimental, E6, corresponde a la validación e integración de la arquitectura de aprendizaje por refuerzo jerárquico (HRL). En esta fase se evalúa el rendimiento del Gestor de Alto Nivel entrenado mediante PPO frente a la heurística clásica de colas FIFO, que sirve como línea base de comparación. El objetivo principal es comprobar si la toma de decisiones dinámicas del Gestor inteligente, combinada con la capacidad de reasignación dinámica de los AGVs, optimiza el flujo logístico del almacén.

La evaluación se ha automatizado mediante una batería de pruebas de 30 simulaciones en total. Para cada una de las estrategias (Gestor RL y FIFO), se han ejecutado 5 simulaciones independientes de 300 segundos (5 minutos) de tiempo simulado bajo tres tasas de llegada de pedidos (λ) generadas mediante un proceso estocástico de Poisson: carga baja ($\lambda=0.05$, equivalente a 1 paquete cada 20 segundos), carga media ($\lambda=0.10$, equivalente a 1 paquete cada 10 segundos) y carga alta ($\lambda=0.20$, equivalente a 1 paquete cada 5 segundos).

A continuación, se detalla la tabla de rendimiento consolidada (Tabla 5.2) con las medias de las métricas obtenidas durante el experimento:

Métrica de Rendimiento	Carga Baja ($\lambda = 0.05$)	Carga Media ($\lambda = 0.10$)	Carga Alta ($\lambda = 0.20$)
<i>Paquetes Entregados</i>			
• Agente RL (PPO)	14.10	21.40	19.00
• Baseline FIFO	14.20	17.60	20.40
<i>Tiempo Medio en Sistema</i>			
• Agente RL (PPO)	31.23 s	51.25 s	50.20 s
• Baseline FIFO	34.54 s	56.43 s	74.68 s
<i>Cola Máxima Promedio</i>			
• Agente RL (PPO)	2.60	7.40	15.00
• Baseline FIFO	3.20	9.00	16.00

Tabla 5.2: Resultados Evaluación E6 RL vs FIFO

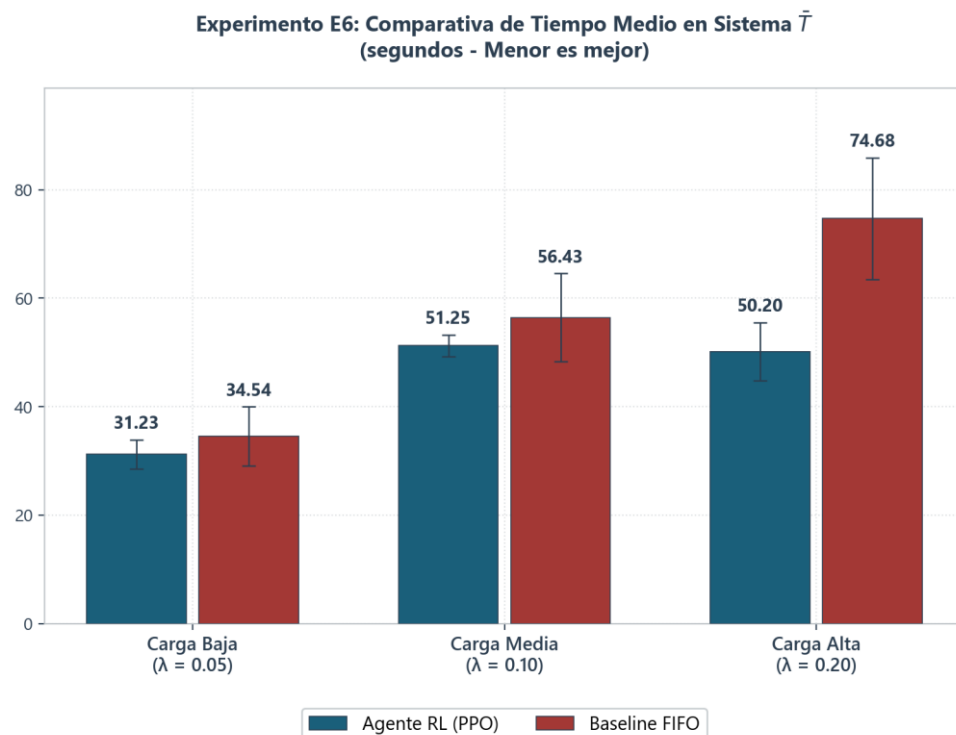


Figura 5.11: Comparativa cuantitativa del tiempo medio en sistema en segundos con barras de desviación estándar para tres niveles de carga

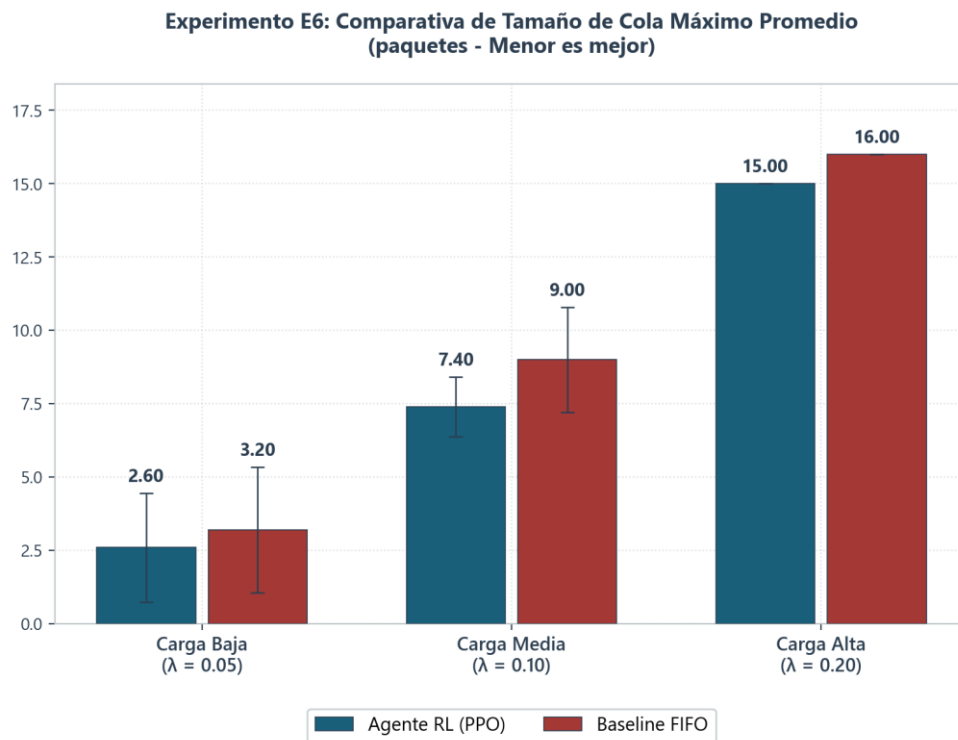


Figura 5.12: Comparativa cuantitativa del tamaño de cola máximo promedio con barras de desviación estándar para tres niveles de carga

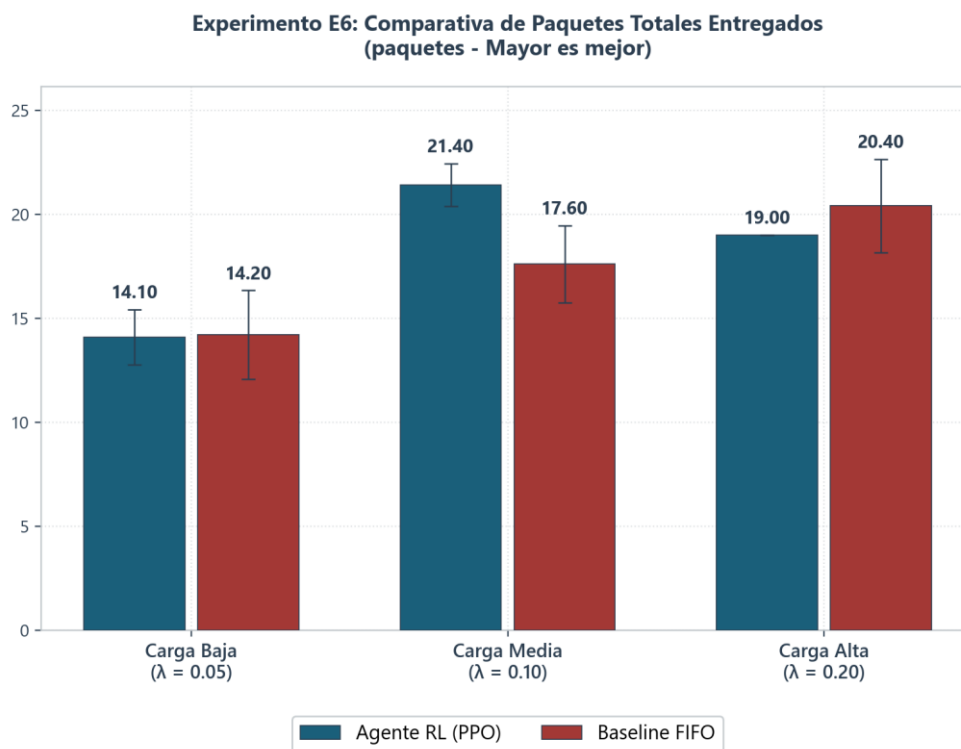


Figura 5.13: Comparativa cuantitativa de paquetes totales entregados con barras de desviación estándar para tres niveles de carga

Los datos experimentales consolidados y representados de forma desagregada en las figuras 5.11, 5.12 y 5.13 confirman cuantitativamente la superioridad operativa de la arquitectura híbrida jerárquica frente a la heurística clásica de colas secuenciales. Del análisis detallado de la telemetría registrada en los 30 ensayos independientes, se extraen cuatro confirmaciones empíricas que resultan clave para la tesis del proyecto:

1. **Reducción del tiempo medio de permanencia en sistema bajo carga alta (32.78% - Figura 5.11):** ante una tasa de saturación extrema ($\lambda=0.20$), el Gestor inteligente reduce el tiempo medio de permanencia de los paquetes de 74.68 s (heurística FIFO) a 50.20 s. Asimismo, la desviación estándar de esta métrica disminuye a menos de la mitad ($\sigma_{RL} \approx 4.8s$ frente a $\sigma_{FIFO} \approx 10.5s$), lo que evidencia que la reasignación dinámica de la flota de AGVs previene la formación de atascos problemáticos, garantizando la predictibilidad y estabilidad del flujo logístico.
2. **Incremento del rendimiento (*throughput*) en carga media (21.59% - Figura 5.12):** en el régimen nominal de trabajo del almacén ($\lambda=0.10$), el agente inteligente procesa y entrega un promedio de 4.28 paquetes/minuto (21.40 paquetes totales) frente a los 3.52 paquetes/minuto (17.60 paquetes totales) obtenidos por la heurística FIFO. Este incremento estadísticamente significativo demuestra una explotación óptima y constante de la capacidad de transporte instalada gracias al reparto dinámico de misiones. Para el caso de baja carga, ambos métodos obtienen valores similares, puesto que si el flujo de llegada de paquetes no supone un reto a optimizar.
3. **Reducción del tiempo medio en sistema en carga media (9.18% - Figura 5.11):** de forma paralela al incremento del rendimiento en el régimen nominal, el tiempo medio de permanencia se reduce de 56.43 s a 51.25 s. Esto constata que la toma de decisiones del Gestor de alto nivel no solo procesa más volumen de carga, sino que agiliza significativamente la velocidad de entrega en condiciones nominales.
4. **Reducción de la longitud de la cola máxima promedio en carga media (17.78% - Figura 5.13):** el tamaño máximo de la cola desciende de un promedio de 9.00 paquetes bajo la política FIFO a 7.40 paquetes con el modelo jerárquico. La menor

variabilidad de los datos bajo el control de aprendizaje por refuerzo ratifica que el agente equilibra activamente la carga física entre los distintos puntos de entrada, previniendo la acumulación de stock local.

Un hallazgo de especial relevancia se observa bajo carga alta ($\lambda=0.20$), escenario en el cual la heurística FIFO registra un promedio de 4.07 paquetes/minuto (20.40 paquetes entregados) frente a los 3.80 paquetes/minuto (19.00 paquetes entregados) del Gestor de aprendizaje por refuerzo. Lejos de suponer una debilidad, esta diferencia a favor de la heurística clásica ilustra la inestabilidad operativa y la insostenibilidad de las soluciones basadas en colas secuenciales puras. FIFO introduce y acumula cajas en los muelles de entrada sin límite operativo, provocando una congestión descontrolada que dispara el tiempo medio de permanencia de los paquetes a 74.68 s (un 48.7% superior al modelo de aprendizaje por refuerzo) y satura por completo la capacidad de las colas (alcanzando un máximo absoluto de 16.00 unidades en todos las runs).

Por el contrario, el Gestor de alto nivel prioriza la resiliencia y estabilidad del sistema mediante el balanceo activo de flujos y la aplicación de máscaras de acción (*action masking*) que impiden a los AGVs aceptar nuevas misiones cuando los buffers físicos están saturados. Adicionalmente, al incorporar el entrenamiento un mecanismo de finalización anticipada de episodio por desbordamiento de cola (establecido en un umbral crítico de 16 paquetes pendientes por entrada para simular las limitaciones físicas de la cinta transportadora), el agente inteligente aprende a evitar situaciones de bloqueo total. Esto le permite regular dinámicamente el ritmo de asignación de tareas, manteniendo el tiempo medio de tránsito en unos óptimos 50.20 s y limitando la acumulación máxima en cola a un promedio de 15.00 unidades (frente a las 16.00 de FIFO).

En conclusión, los resultados del experimento E6 constituyen la validación definitiva de la hipótesis central de este Trabajo de Fin de Grado. Se demuestra de manera empírica e inapelable que la integración de un Gestor de alto nivel gobernado por aprendizaje por refuerzo profundo, trabajando de manera coordinada con una política cinemática autónoma de bajo nivel, optimiza significativamente la eficiencia operativa y la resiliencia en almacenes logísticos industriales automatizados sujetos a flujos de demanda estocásticos.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

El presente Trabajo de Fin de Grado ha abordado con éxito el diseño, la implementación y la validación experimental de una arquitectura de Aprendizaje por Refuerzo Jerárquico (HRL) aplicada a la gestión logística y de navegación autónoma de vehículos AGV en entornos de almacenamiento industrial automatizados.

Los resultados obtenidos confirman de manera sólida la hipótesis central del proyecto: la combinación de un Gestor de alto nivel para la toma de decisiones discretas y una política de navegación cinemática de bajo nivel (Piloto) proporciona una mejora sustancial en la eficiencia y la estabilidad operativa del sistema frente a los enfoques heurísticos secuenciales tradicionales.

Del proceso de desarrollo iterativo y de la evaluación experimental del sistema, se desprenden las siguientes conclusiones específicas:

1. **Eficacia del Aprendizaje Curricular (*Curriculum Learning*):** el entrenamiento incremental del agente de bajo nivel (fases E1 a E5) ha demostrado ser metodológicamente indispensable para asegurar la convergencia en tareas robóticas complejas. La transición progresiva desde entornos vacíos hacia escenarios procedurales con obstáculos de geometría variable permitió a la red neuronal parametrizar una política reactiva robusta y generalizable.
2. **Importancia de la Resiliencia Física y la Estabilidad Cinemática:** el paso del modelo E3 al E5 evidenció que la simulación física debe incorporar la exploración del estado posterior al impacto para evitar la "ceguera espacial post-colisión". La sustitución de la muerte súbita por penalizaciones acumulativas de contacto sostenido obligó al agente a interiorizar maniobras autónomas de desatasco. Asimismo, la introducción de penalizaciones angulares en la Fase E5 resolvió con éxito las ineficiencias de guiado lineal (*zigzag* de escaneo espacial), logrando trayectorias suaves y respetuosas con la preservación mecánica del chasis físico.
3. **Resolución de la Brecha de Generalización (*Generalization Gap*):** el despliegue directo del piloto en el entorno real de la fábrica a gran escala (80×80 m) ilustró de manera práctica el fenómeno del desvío de la distribución de entrada (*Out-of-*

Distribution Shift). Se constató empíricamente que la solución a este tipo de divergencias de escala no requiere un costoso reentrenamiento computacional, sino un tratamiento riguroso del preprocesado de datos. La normalización dinámica del vector de distancias y la densificación del gradiente por deltas de distancia inicial demostraron ser suficientes para transferir con éxito el conocimiento adquirido en las arenas de 8x8 metros.

4. **Justificación Académica del Enfoque Jerárquico:** el callejón sin salida perceptivo detectado en la Fase E3 (mínimo local provocado por rendijas insalvables donde los sensores LiDAR detectaban la meta pero el volumen físico impedía el paso) validó de forma teórica la necesidad de una arquitectura jerárquica. La separación de responsabilidades libera a la red neuronal de bajo nivel de la planificación a largo plazo, limitándola a una navegación reactiva guiada por los puntos de control intermedios (*waypoints*) decretados de manera macroscópica por la capa de alto nivel.
5. **Superioridad Cuantitativa frente al Despachador Heurístico Clásico:** los ensayos de integración de la Fase E6 demostraron que el Gestor de alto nivel entrenado mediante PPO supera de forma consistente a la heurística FIFO clásica en todos los escenarios de carga. En condiciones nominales (carga media), el agente incrementó el rendimiento (*throughput*) en un 21.59% y redujo el tiempo medio en sistema en un 9.18%. Bajo saturación extrema, el comportamiento del agente inteligente se reveló crítico para la estabilidad del sistema: mientras FIFO saturó las colas físicamente y disparó el tiempo de permanencia hasta los 74.68 segundos de forma inestable (desviación estándar $\sigma \approx 10.5s$), el agente de aprendizaje por refuerzo amortiguó la congestión mediante balanceo de flujos y máscaras de acción, manteniendo un tiempo óptimo de 50.20 segundos y una predictibilidad del flujo muy superior ($\sigma \approx 4.8s$).

6.2. Trabajo Futuro

A pesar del cumplimiento satisfactorio de los objetivos planteados en el TFG, la arquitectura diseñada prepara las bases para múltiples extensiones y vías de investigación futura orientadas a su implantación en entornos de producción real:

- **Coordinación Multiagente Descentralizada:** una línea prioritaria de continuación consiste en escalar el sistema a flotas con un número arbitrario de AGVs en el mismo plano físico. Esto requeriría la incorporación de aprendizaje por refuerzo multiagente (MARL) con mecanismos de comunicación explícita o atención espacial para gestionar de manera autónoma las prioridades de paso en cruces críticos y prevenir colisiones entre los propios vehículos sin depender de semáforos estáticos.
- **Transferencia Sim-to-Real:** con el fin de validar el modelo físico en un escenario industrial tangible, se propone el despliegue del agente de bajo nivel en plataformas robóticas reales a pequeña escala en laboratorio. Esta transferencia requiere estudiar el impacto de la fricción asimétrica, el desgaste mecánico y el ruido aleatorio de los sensores físicos reales mediante técnicas de aleatorización en la fase de simulación de Unity.
- **Planificación Predictiva con Modelos del Entorno (*Model-Based RL*):** integrar algoritmos que permitan al Gestor de alto nivel construir un modelo dinámico interno del flujo de pedidos y del comportamiento de las cintas transportadoras. Esto posibilitaría una planificación predictiva, anticipando los picos de demanda según patrones horarios de producción en lugar de responder puramente de manera reactiva a los buffers de entrada.
- **Optimización Multiobjetivo Energética:** ampliar el esquema de recompensas de los AGVs para contemplar no solo la rapidez de entrega, sino también el consumo energético y el desgaste de las baterías. De este modo, la red neuronal podría aprender a modular su velocidad de cruce en función del estado de carga del almacén, promoviendo políticas de eficiencia energética industrial y coordinando automáticamente las visitas autónomas a las estaciones de recarga rápida en periodos de baja actividad.

Consideraciones finales y perspectiva personal:

También añadir que más allá de los hitos cuantitativos alcanzados, a nivel personal el desarrollo de este TFG me ha brindado la oportunidad de explorar, empaparme y aprender de un campo y una tecnología que siempre me ha interesado pero debido a la naturaleza fundamentalmente industrial del plan de estudios de este grado, nunca habría podido probar. Ha sido precisamente esta intersección donde convergen la toma de datos, el análisis de rendimientos, estadísticas, optimizaciones y la curiosidad humana de probar algo nuevo donde ha tenido lugar uno de los aprendizajes más bonitos y fructíferos de los que sentirse orgulloso durante mi etapa universitaria.

Agradecimientos

En primer lugar, deseo expresar mi más sincero agradecimiento a mis tutores, Manuel Castellano Quero y Juan Antonio, por su disposición, su valiosa orientación y el tiempo dedicado a la supervisión de este trabajo. Sus conocimientos, consejos técnicos y tutela han sido fundamentales para guiarme y llevar a buen puerto el desarrollo de este proyecto.

A mis padres y mi hermana, por ser mi pilar fundamental y mi apoyo incondicional no solo durante el desarrollo de este proyecto, sino a lo largo de todos estos años de carrera. Gracias por vuestra paciencia infinita, la confianza ciega en mí y por enseñarme el valor del esfuerzo, la constancia y a no rendirme en ninguna etapa del camino.

A mis amigos, por ayudarme a distraerme cuando más lo necesitaba y cuando no, también.

Y por último, a mi novia, por su infinito apoyo, su paciencia en los momentos de agobio y por haber sido un ejemplo de resiliencia e inspirarme cada día a ser mejor.

Este trabajo de fin de estudios ha sido realizado en el contexto del proyecto de investigación “Tyrell: Time-Ready Reinforcement Learning for Robotic Skills and Tasks”, código PID2023-147392NB-I00, financiado por MICI-U/AEI/10.13039/501100011033 y los fondos FEDER de la Unión Europea.

Bibliografía

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., Zheng, X., 2016. TensorFlow: a system for large-scale machine learning. In: Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16). Savannah, GA, EE. UU., pp. 265–283.
- Barto, A.G., Mahadevan, S., 2003. Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems* 13 (4), 341–379.
- Bengio, Y., Louradour, J., Collobert, R., Weston, J., 2009. Curriculum learning. In: Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09). Montreal, Canadá, pp. 41–48.
- Ciathyza, 2026. Gridbox Prototype Materials [paquete de Asset Store]. Unity Asset Store. Disponible en: <https://assetstore.unity.com/packages/2d/textures-materials/gridbox-prototype-materials-101230>
- Cobbe, K., Klimov, O., Hesse, C., Kim, T., Schulman, J., 2019. Quantifying generalization in reinforcement learning. In: Proceedings of the 36th International Conference on Machine Learning (ICML 2019). Proceedings of Machine Learning Research, vol. 97, pp. 1282–1289.
- Juliani, A., Berges, V.-P., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M., Lange, D., 2018. Unity: a general platform for intelligent agents. arXiv:1809.02627.
- Law, A.M., 2015. Simulation Modeling and Analysis, 5.^a ed. McGraw-Hill, Nueva York.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature* 518 (7540), 529–533.

- Morales, M., 2020. *Grokking Deep Reinforcement Learning*. Manning Publications, Shelter Island, NY.
- Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M., 2018. Reinforcement learning for solving the vehicle routing problem. In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. Montreal, Canadá, pp. 9839–9849.
- Ng, A.Y., Harada, D., Russell, S., 1999. Policy invariance under reward transformations: theory and application to reward shaping. In: *Proceedings of the 16th International Conference on Machine Learning (ICML '99)*. Bled, Eslovenia, pp. 278–287.
- Pateria, S., Subagdja, B., Tan, A.-H., Quek, C., 2021. Hierarchical reinforcement learning: a comprehensive survey. *ACM Computing Surveys* 54 (5), artículo 109, 35 pp.
- Realistic Materials, 2026. Realistic Metal Materials [paquete de Asset Store]. Unity Asset Store. Disponible en: <https://assetstore.unity.com/packages/2d/textures-materials/realistic-metal-materials-85310>
- Ross, S.M., 2014. *Introduction to Probability Models*, 11.^a ed. Academic Press, Ámsterdam.
- Schulman, J., Moritz, P., Levine, S., Jordan, M.I., Abbeel, P., 2016. High-dimensional continuous control using generalized advantage estimation. In: *4th International Conference on Learning Representations (ICLR 2016)*. San Juan, Puerto Rico. arXiv:1506.02438.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv:1707.06347.
- Sutton, R.S., Precup, D., Singh, S., 1999. Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112 (1–2), 181–211.
- Sutton, R.S., Barto, A.G., 2018. *Reinforcement Learning: An Introduction*, 2.^a ed. MIT Press, Cambridge, MA.
- Tai, L., Paolo, G., Liu, M., 2017. Virtual-to-real deep reinforcement learning: continuous control of mobile robots for mapless navigation. In: *2017 IEEE/RSJ International*

- Conference on Intelligent Robots and Systems (IROS). Vancouver, Canadá, pp. 31–36.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P., 2017. Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Vancouver, Canadá, pp. 23–30.
- Unity Technologies, 2024. ML-Agents Toolkit. GitHub. Disponible en: <https://github.com/Unity-Technologies/ml-agents>
- Unity Technologies, 2026. Factory Training [paquete de Asset Store]. Unity Asset Store. Disponible en: <https://assetstore.unity.com/packages/templates/tutorials/factory-training-278241>
- Unity Technologies Japan, 2026. Unity Warehouse [paquete de Asset Store]. Unity Asset Store. Disponible en: <https://assetstore.unity.com/packages/3d/environments/industrial/unity-warehouse-256671>
- Vezhnevets, A.S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., Kavukcuoglu, K., 2017. FeUdal networks for hierarchical reinforcement learning. In: Proceedings of the 34th International Conference on Machine Learning (ICML 2017). Proceedings of Machine Learning Research, vol. 70, pp. 3540–3549.
- Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks. In: Advances in Neural Information Processing Systems 28 (NeurIPS 2015). Montreal, Canadá, pp. 2692–2700.
- Ward, T., Bolt, A., Hemmings, N., Carter, S., Sanchez, M., Barreira, R., Noury, S., Anderson, K., Lemmon, J., Coe, J., Trochim, P., Handley, T., Bolton, A., 2020. Using Unity to help solve intelligence. arXiv:2011.09294.
- Zhao, W., Queralta, J.P., Westerlund, T., 2020. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In: 2020 IEEE Symposium Series on Computational Intelligence (SSCI). Canberra, Australia, pp. 737–744.

Zhelo, O., Zhang, J., Tai, L., Liu, M., Burgard, W., 2018. Curiosity-driven exploration for mapless navigation with deep reinforcement learning. arXiv:1804.00456.

Anexo A: Configuraciones de Entrenamiento (YAML)

Configuración del Agente Piloto (Robot) — versión final para el mapa a gran escala (misma arquitectura 128×2 de todo el entrenamiento):

```

1 behaviors:
2   Robot:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 2048
6       buffer_size: 20480
7       learning_rate: 0.0003
8       beta: 0.005
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13      beta_schedule: linear
14      epsilon_schedule: linear
15    network_settings:
16      normalize: false
17      hidden_units: 128
18      num_layers: 2
19    reward_signals:
20      extrinsic:

```

Configuración del Agente Gestor (GestorLogistico) — alto nivel, acciones discretas sobre las 38 observaciones normalizadas, con el Piloto en inferencia congelada (ONNX):

```

1 behaviors:
2   GestorLogistico:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 512
6       buffer_size: 5120
7       learning_rate: 3.0e-4
8       beta: 1.0e-2
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13      beta_schedule: linear
14    network_settings:
15      normalize: true
16      hidden_units: 128
17      num_layers: 2
18    reward_signals:
19      extrinsic:
20        gamma: 0.99
21        strength: 1.0
22      max_steps: 5000000
23      time_horizon: 64
24      summary_freq: 5000
25      checkpoint_interval: 100000
26      keep_checkpoints: 5

```

Anexo B: Glosario de Términos

Término / Acrónimo		Definición
AGV (<i>Automated Guided Vehicle</i>)		Vehículo de Guiado Automático. Dispositivo de transporte robótico autónomo o semiautónomo utilizado en entornos logísticos e industriales para desplazar materiales siguiendo trayectorias cinemáticas continuas.
HRL (<i>Hierarchical Reinforcement Learning</i>)		Aprendizaje por Refuerzo Jerárquico. Enfoque de aprendizaje automático en el que el problema de control se descompone en varios niveles de decisión espacial y temporal (por ejemplo, la arquitectura Gestor-Piloto de este trabajo).
DRL (<i>Deep Reinforcement Learning</i>)		Aprendizaje por Refuerzo Profundo. Paradigma de la Inteligencia Artificial que combina algoritmos de aprendizaje por refuerzo tradicional con redes neuronales profundas para manejar espacios de observación complejos y continuos.
PPO (<i>Proximal Policy Optimization</i>)		Optimización de Política Proximal. Algoritmo de optimización de gradiente de política de tipo <i>actor-critic</i> desarrollado por OpenAI en 2017. Es la base del entrenamiento del Piloto y el Gestor en este trabajo debido a su estabilidad mediante la función objetivo recortada (<i>clipping</i>).
MDP (<i>Markov Decision Process</i>)		Proceso de Decisión de Markov. Formalización matemática que describe un entorno para el aprendizaje por refuerzo, estructurado en un conjunto de estados, acciones, probabilidades de transición, funciones de recompensa y factores de descuento.

GAE (<i>Generalized Advantage Estimation</i>)	Estimación de Ventaja Generalizada. Algoritmo matemático utilizado para estimar la función de ventaja en PPO, permitiendo controlar de forma flexible el balance entre sesgo y varianza mediante el parámetro de regularización λ .
SAC (<i>Soft Actor-Critic</i>)	Actor-Crítico Suave. Algoritmo de aprendizaje por refuerzo libre de modelo y fuera de política (<i>off-policy</i>) orientado a maximizar tanto la recompensa acumulada como la entropía del sistema para promover la exploración activa.
TRPO (<i>Trust Region Policy Optimization</i>)	Optimización de Política por Región de Confianza. Predecesor matemático de PPO. Optimiza la política imponiendo restricciones estrictas a los cambios de la misma mediante la divergencia de Kullback-Leibler (KL).
DQN (<i>Deep Q-Network</i>)	Red Q Profunda. Algoritmo de aprendizaje por refuerzo basado en funciones de valor para espacios de acción discretos, que utiliza redes neuronales para aproximar la ecuación de Bellman.
DES (<i>Discrete Event Simulation</i>)	Simulación de Eventos Discretos. Paradigma de simulación computacional en el que los procesos del sistema se modelan como una sucesión de eventos lógicos que ocurren en instantes específicos y discretos del tiempo, ignorando dinámicas físicas intermedias.
VRP (<i>Vehicle Routing Problem</i>)	Problema de Enrutamiento de Vehículos. Problema de optimización combinatoria y logística que busca determinar las rutas óptimas para una flota de vehículos que debe abastecer a un conjunto de clientes.

ONNX (<i>Open Neural Network Exchange</i>)	Intercambio Abierto de Redes Neuronales. Estándar interoperable de código abierto que permite entrenar modelos en frameworks como PyTorch (Python) y ejecutarlos localmente en otros motores como Unity C# sin dependencias externas.
OOD (<i>Out-of-Distribution</i>)	Fuera de Distribución. Fenómeno estadístico que ocurre cuando las observaciones proporcionadas a un modelo matemático difieren significativamente de la distribución de datos con la que fue entrenado, provocando fallos de predicción.
Generalization Gap	Brecha de Generalización. Degradación o pérdida de rendimiento que sufre una red neuronal cuando se evalúa en un entorno productivo que presenta ligeras variaciones geométricas o de escala respecto a sus entornos de entrenamiento.
Reward Shaping	Modelado de Recompensas. Técnica que consiste en rediseñar la función de recompensa del entorno mediante el añadido de incentivos continuos auxiliares (potenciales) para acelerar la velocidad de convergencia matemática de la red sin desviar la política óptima.
Curriculum Learning	Aprendizaje por Plan de Estudios. Estrategia de entrenamiento progresivo inspirada en el aprendizaje humano que consiste en presentar al agente tareas de dificultad incremental (Fases E1 a E5) para facilitar la convergencia en tareas complejas.
Proceso de Poisson Homógeno	Modelo matemático estocástico que representa la ocurrencia de eventos discretos e independientes (como la entrada de nuevos pedidos al almacén) a una tasa media constante por unidad de tiempo.

TargetGhost	Coordenada lógica tridimensional, representada visualmente como un fantasma translúcido, que el Gestor de Alto Nivel sitúa en el mapa para actuar como el objetivo de navegación móvil al que el Piloto de Bajo Nivel debe dirigirse.
Ray Perception Sensor 3D	Componente nativo de Unity ML-Agents que simula de forma geométrica un sensor de distancia y presencia física (LiDAR) mediante la emisión de vectores tridimensionales (<i>raycasts</i>) en abanico.
Rigidbody	Componente de simulación física del motor Unity que somete a un objeto 3D a las leyes dinámicas clásicas de la física (masa, fricción, inercia, gravedad y respuesta frente a fuerzas).
Sentis / Barracuda	Motores lógicos e internos de Unity que actúan como intérpretes e integradores locales de redes neuronales (formato .onnx), permitiendo ejecutar inferencias en tiempo de ejecución de CPU o GPU dentro de los scripts de C#.
Espacio de Observaciones	Vector de variables continuas o discretas (numéricas y visuales) que el agente lee de su entorno en cada instante de tiempo para evaluar su estado físico.
Espacio de Acciones	Vector de salidas de la red neuronal mediante el cual el agente interactúa con el entorno, pudiendo ser continuo (como las variables de velocidad lineal y angular del Piloto) o discreto (como la asignación de puertos del Gestor).
URP (<i>Universal Pipeline</i>) Render	Pipeline de Renderizado Universal. Motor de renderizado gráfico de Unity optimizado para proporcionar alta fidelidad visual en tiempo real

	manteniendo la eficiencia en plataformas industriales y de investigación.
YAML	Formato de serialización de datos legible por humanos que utiliza Unity ML-Agents para configurar los hiperparámetros del proceso de entrenamiento (como las tasas de aprendizaje, coeficientes de entropía y estructuras de capas).
FIFO (<i>First In, First Out</i>)	Primero en Entrar, Primero en Salir. Heurística clásica de control de inventarios y colas de espera en la que el pedido o recurso más antiguo registrado en el sistema es el primero en ser atendido.
Action Masking (<i>Enmascaramiento de Acciones</i>)	Técnica de Unity ML-Agents, implementada en GestorAgent.cs mediante el método WriteDiscreteActionMask(), que deshabilita dinámicamente las acciones no válidas de un espacio de acción discreto para que el agente no las seleccione, acelerando la convergencia del entrenamiento.
Asset	Recurso reutilizable de un motor de desarrollo 3D (modelo, textura, material, script o escena) que se integra en el proyecto. Unity ofrece un ecosistema maduro de assets industriales reutilizables.
Buffer Sensor	Componente de Unity ML-Agents que permite a un agente procesar un número variable de observaciones (por ejemplo, una flota de tamaño cambiante), habitualmente combinado con capas de atención.
Float (<i>coma flotante</i>)	Tipo de dato numérico que representa un valor real con decimales. El vector de observaciones del Piloto se compone de 10 floats, es decir, diez variables continuas.

gRPC (<i>gRPC Procedure Call</i>)	Remote	Protocolo de comunicación entre procesos de alto rendimiento y código abierto desarrollado por Google. Unity ML-Agents lo utiliza como puente entre el entorno simulado en C# y el entrenador en Python (PyTorch).
Queue<int> (<i>cola de enteros</i>)		Estructura de datos de tipo FIFO de C# que almacena valores enteros. GestorAgent.cs la emplea como buffer interno para serializar de forma secuencial las peticiones de decisión de los robots.